Sustavi baza podataka

Predavanja

2. Fizička organizacija

ožujak 2026.

Fizička organizacija

- fizička organizacija podataka ima vrlo veliki utjecaj na efikasnost (*efficiency*) sustava za upravljanje bazama podataka
- pojam fizičke organizacije podataka odnosi se na:
  - organizacija datoteke (*file organization*): strukture podataka primijenjene pri pohrani podataka u sekundarnoj memoriji
  - metode pristupa (*access methods*): postupci koji se primjenjuju pri obavljanju operacija nad podacima
- primijenjenim strukturama podataka u osnovi se određuje
  - u koji fizički blok pohraniti logički zapis odnosno n-torku
  - koje je dodatne informacije potrebno pohraniti da bi se omogućio efikasni pristup podacima
- primjenjivost pojedinih metoda ovisi o primijenjenim strukturama podataka
  - za pohranu sadržaja relacije nastojati primijeniti strukturu podataka koje će omogućiti efikasno obavljanje onih operacija koje se nad tom relacijom najčešće obavljaju
  - često je potreban kompromis

Važniji ciljevi fizičke organizacije

- minimizirati broj U/I operacija pri pohrani i dohvatu podataka
- minimizirati utrošak prostora za pohranu
- omogućiti obavljanje različitih tipova upita na efikasan način. Npr.
  - sve n-torke, eventualno sortirano
  - jedna n-torka na temelju ključa pretrage (primarni ili alternativni ključ)
  - grupa n-torki na temelju vrijednosti ključa pretrage, gdje ključ pretrage (*search key*) ne mora nužno biti primarni ili alternativni ključ
  - grupa n-torki iz intervala vrijednosti
  - itd.

Strukture podataka i metode pristupa podacima

- ne postoji "najbolja" metoda fizičke organizacije, ali neke od njih se u današnjim sustavima za upravljanje bazama podataka češće koriste

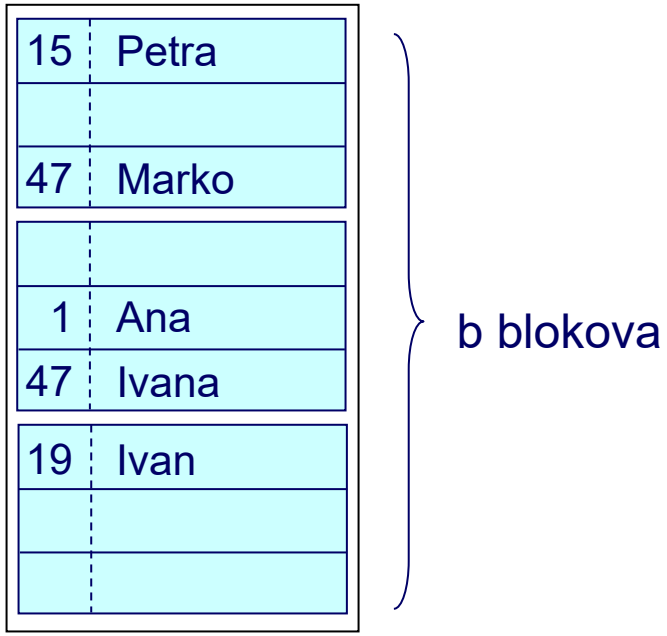
  - **Neporedana datoteka**
  - **Neporedana datoteka + B-stabla**
  - **B-stablo s n-torkama u listovima stabla**

  - **Raspršena datoteka**
  - Poredana datoteka

Neporedana datoteka

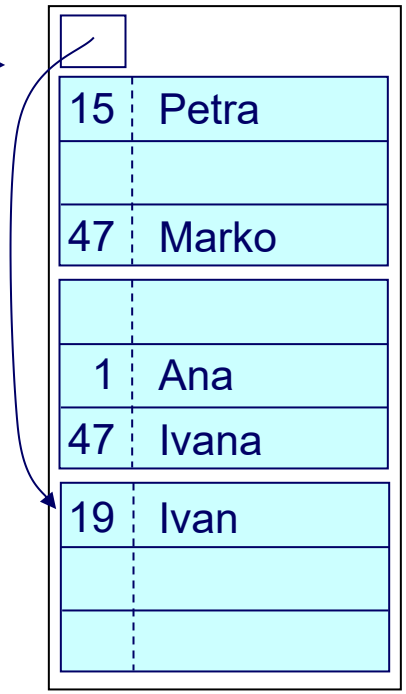
- *heap file*, *pile file*, *file of unordered records*
- pozicija zapisa u datoteci ovisi o redoslijedu unosa
- datoteka sadrži *b* blokova
- pristup podacima (dohvat podatka sa zadanom vrijednošću ključa pretrage) moguć je isključivo linearnim (slijednim) pretraživanjem (*linear search*)

- traženje zapisa sa zadanim ključem
  - ako se radi o primarnom ključu zapisa koji postoji tada je prosječan broj potrebnih U/I operacija *b*/2
  - ako ključ potrage nije primarni ključ, broj obavljenih U/I operacija je *b*



Neporedana datoteka

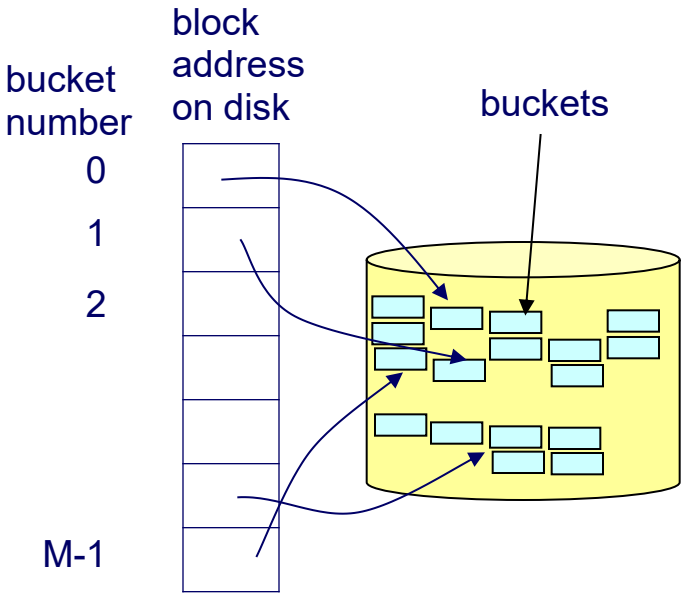
u zaglavlju datoteke (*file header*) nalazi se adresa zadnjeg nepopunjenog bloka



- operacija unosa je vrlo efikasna: zadnji blok datoteke prenosi se u međuspremnik (ako već nije ondje), u blok se upiše novi zapis, blok se zapiše u sekundarnu memoriju (ne nužno odmah)
- brisanje zapisa je najčešće logičko (broj U/I operacija približno kao kod traženja zapisa)

Raspršena datoteka

- *hash files:* vrlo slično raspršenoj organizaciji u primarnoj memoriji
  - *external hashing, secondary storage hashing*
- pretinac (*bucket*): jedan blok ili više uzastopnih blokova u sekundarnoj memoriji
- *hash*-funkcijom se iz ključa izračunava relativna adresa pretinca

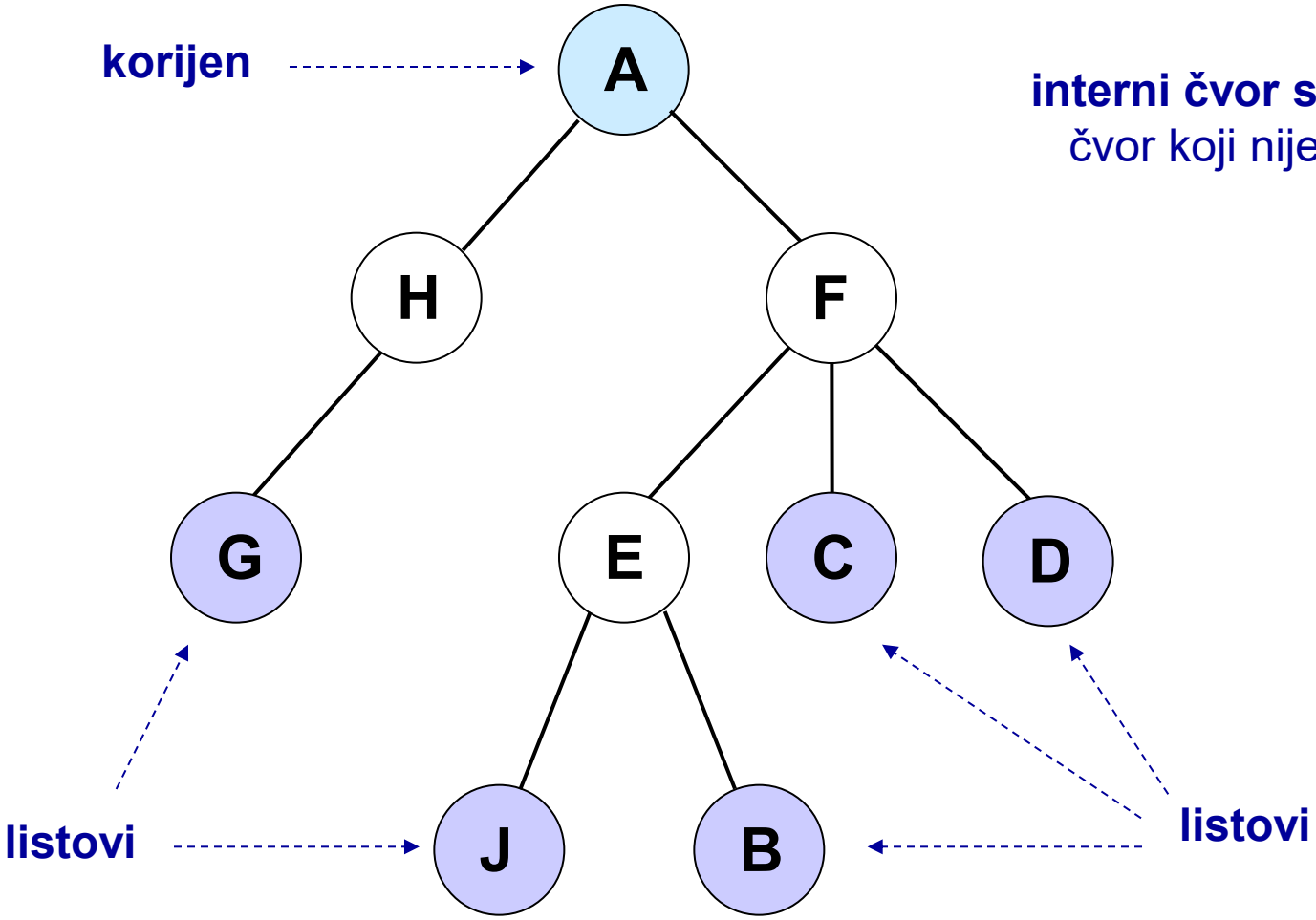


Važna svojstva

- najbrža metoda za operaciju pretraživanja prema ključu (najčešće samo jedna U/I operacija)
- efikasno dodavanje i brisanje zapisa
- problem: eventualni preljevi → ulančavanje
- pretraživanje moguće samo prema ključu (ne intervalu) jer zapisi nisu sortirani

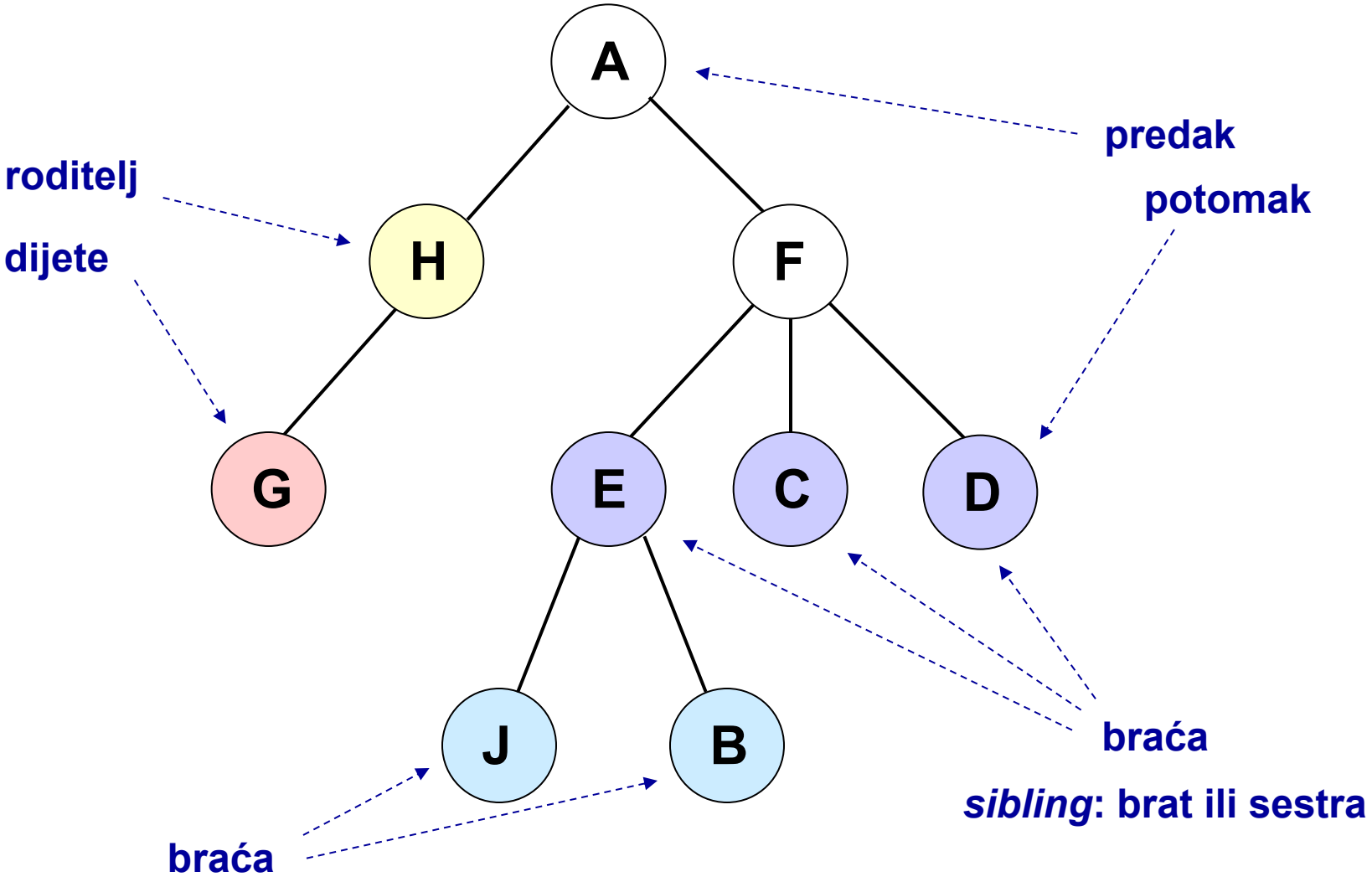
B^+-stablo

Stablo kao struktura podataka

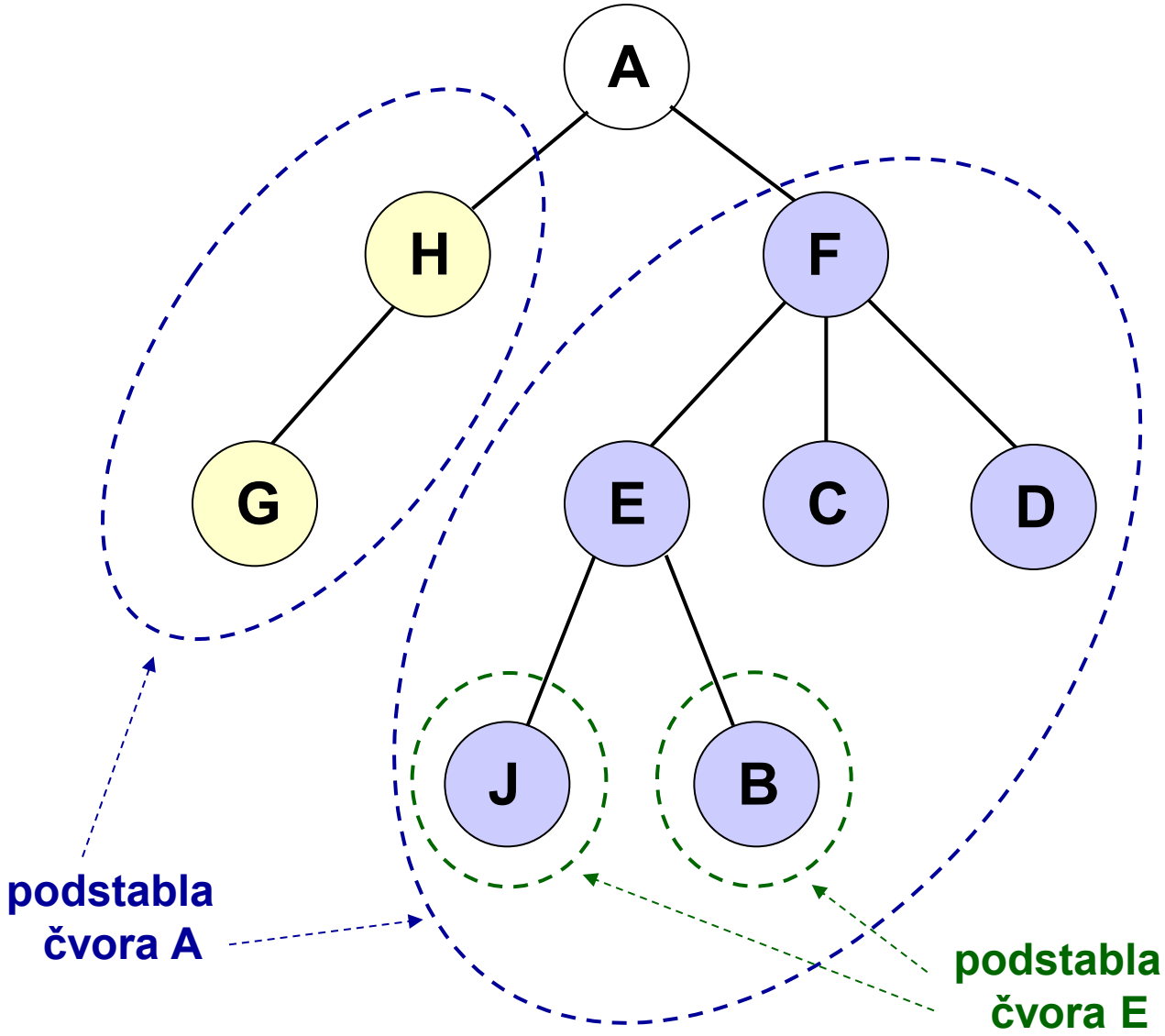


interni čvor stabla: svaki čvor koji nije list

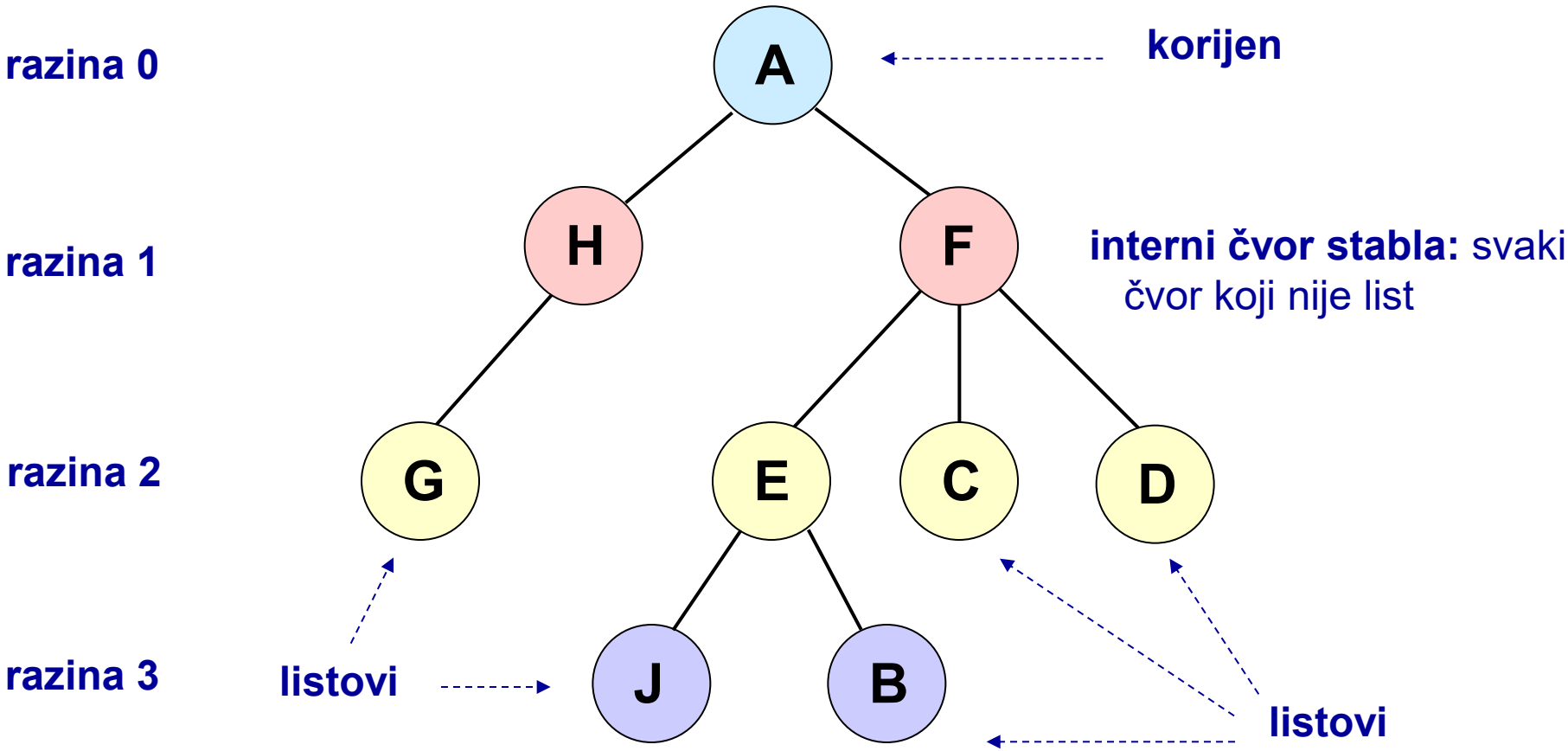
Stablo kao struktura podataka



Stablo kao struktura podataka



Stablo kao struktura podataka

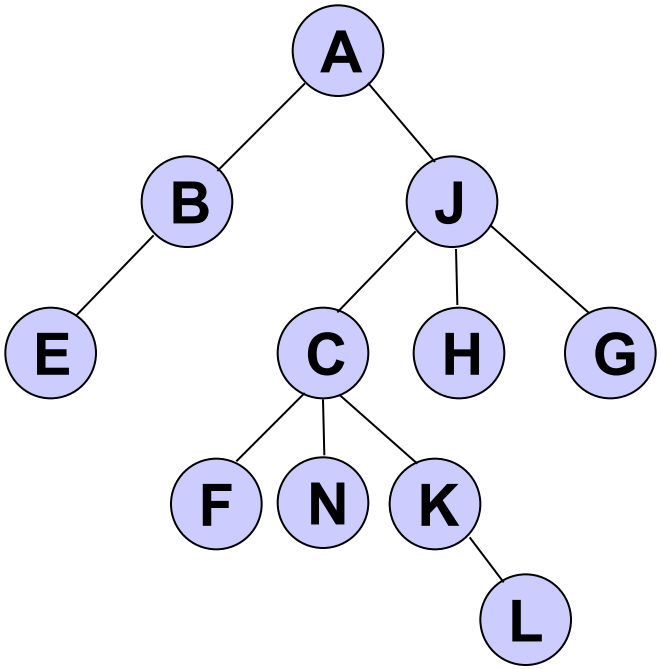


- **razina čvora (*level*):** duljina puta od korijena do čvora
- **dubina stabla (*depth*):** najveća duljina puta od korijena do lista
- **red stabla (*order*):** najveći broj djece koje čvor može imati

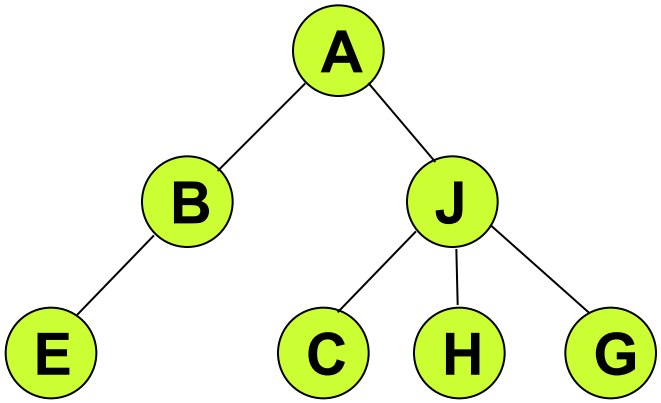
Stablo kao struktura podataka

Stablo je **balansirano (*balanced*)** ako je duljina puta od korijena do lista jednaka za svaki list u stablu

stablo nije balansirano



stablo je balansirano



Oznaka **B** u B-stablo znači **"balansirano"!**

Što je balansirano stablo?

Informativno

O nekonzistentnim i nepreciznim definicijama u literaturi

NIST - National Institute of Standards and Technology / Software and Systems Division / Information Technology Laboratory

Balanced tree: A tree where no leaf is *much farther* away from the root than any other leaf. Different balancing schemes allow different definitions of "much farther".

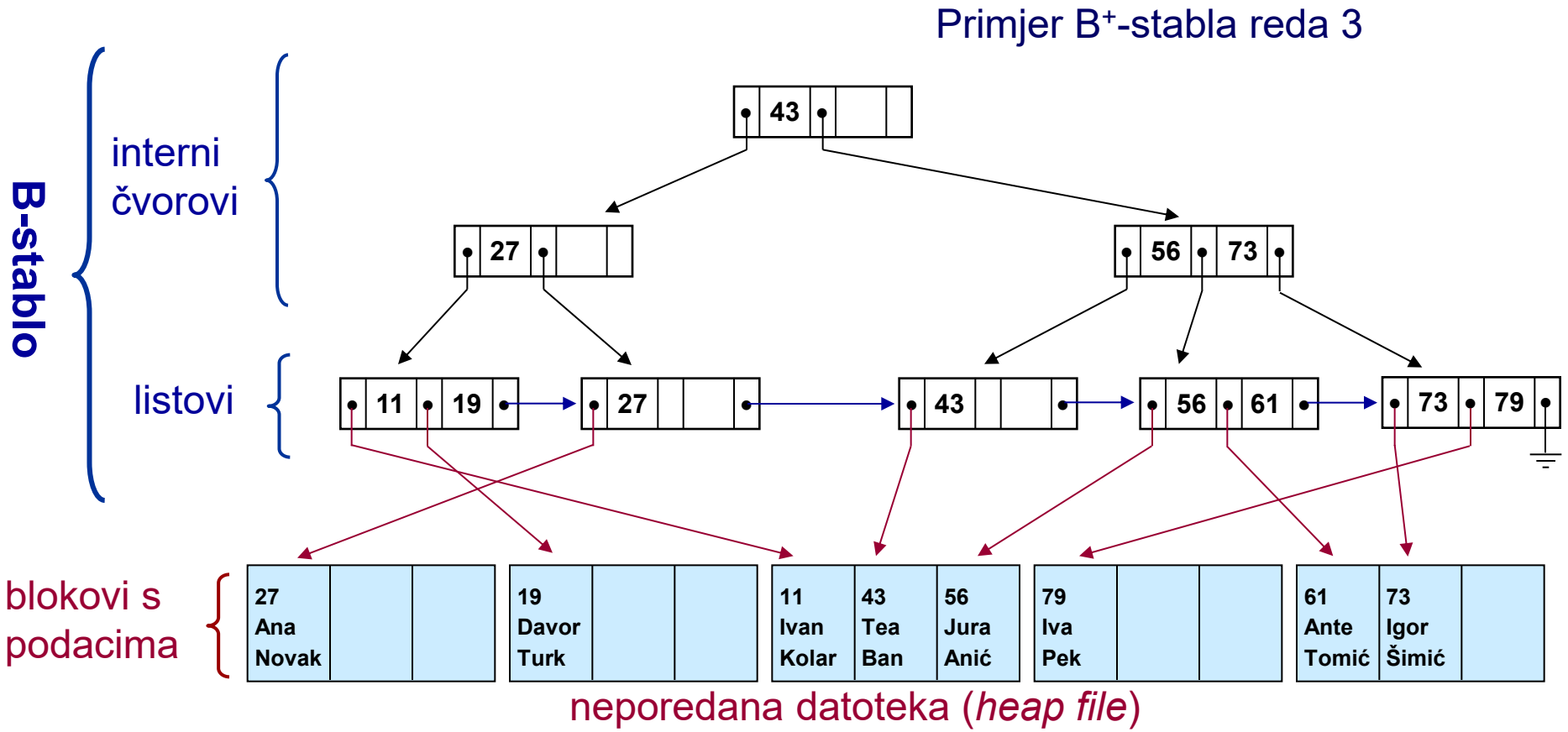
AVL tree: a *balanced binary* search tree where the height of the two subtrees (children) of a node differs by at most one.

R. Elmasri, S. Navathe. Fundamentals of database systems, 3rd ed. Addison-Wesley Publishing Company. Reading, Massachusetts. 2000.

[...] B-tree [...] that a search tree is ***balanced***, meaning that all of its leaf nodes are at the same level.[7]

7. The definition of *balanced* is different for binary trees. Balanced binary trees are known as AVL trees.

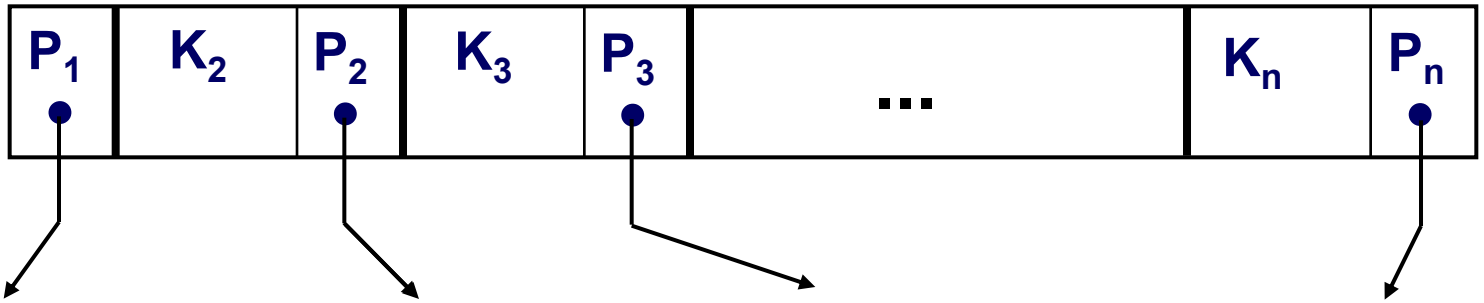
Struktura B⁺-stabla



- Shema: mbr, ime, prezime; B⁺-stablo je izgrađeno za atribut mbr
- Moguće metode pristupa podacima (za bilo koji ključ pretrage):
  - linearnim pretraživanjem (kao kod neporedane datoteke)
  - ako je ključ pretrage mbr, može se koristiti ovo B-stablo

Struktura internog čvora B$^+$-stabla

- U B$^+$-stablu reda **n**, interni čvor sadrži:
  - najviše **n** kazaljki
  - najmanje $\lceil \mathbf{n/2} \rceil$ kazaljki $\rightarrow$ $\lceil \mathbf{a} \rceil$ je najmanji cijeli broj $\geq \mathbf{a}$
    - ovo ograničenje ne vrijedi za korijen (gdje je najmanji broj kazaljki 2)
  - uz **p** kazaljki u čvoru, broj pripadnih vrijednosti K_i u čvoru je **p-1**
    - K_i je vrijednost ključa



podstablo s vrijednostima ključa k **< K_2**

podstablo s vrijednostima ključa k **$K_2 \leq k < K_3$**

podstablo s vrijednostima ključa k **$K_3 \leq k < K_4$**

podstablo s vrijednostima ključa **$k \geq K_n$**

Struktura lista B$^+$-stabla

- U B$^+$-stablu reda **n**, list sadrži:
  - najviše **n-1** vrijednosti K_i i pripadnih kazaljki na zapise (ili blokove)
  - najmanje $\lceil$**(n-1)/2**$\rceil$ vrijednosti K_i i pripadnih kazaljki na zapise
  - svi listovi (osim zadnjeg) sadrže kazaljku na sljedeći list
    - omogućuje upite tipa od-do (prema zadanim granicama intervala)



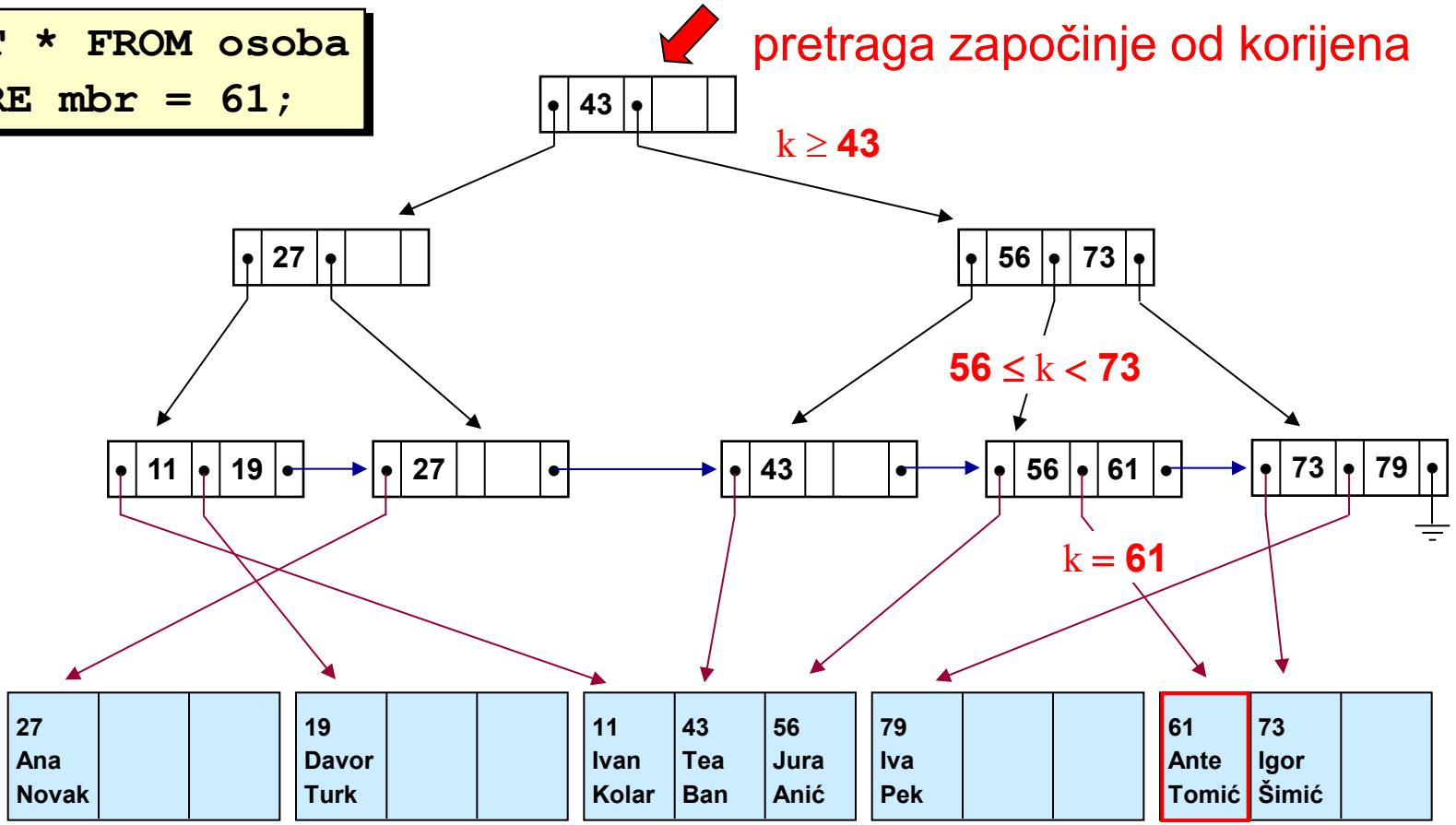
zapis (ili blok koji ga sadrži) s vrijednošću ključa k = **K_1**

zapis (ili blok koji ga sadrži) vrijednošću ključa k = **K_2**

zapis (ili blok koji ga sadrži) s vrijednošću ključa k = **K_{n-1}**

Algoritam za pronalaženje zapisa putem B$^+$-stabla

```
SELECT * FROM osoba
  WHERE mbr = 61;
```



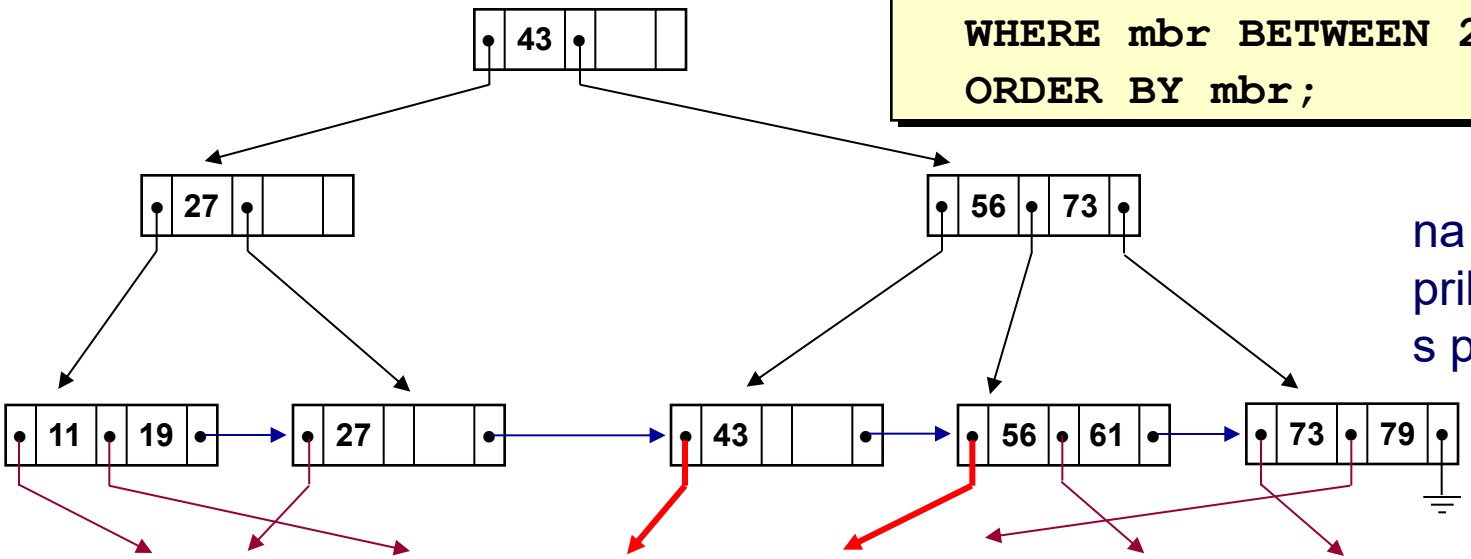
- slijediti odgovarajuću kazaljku do sljedeće razine
- postupak se ponavlja dok se ne dođe do lista u kojem će se naći kazaljka na zapis u bloku s podacima (ako n-torka s tim ključem uopće postoji)

Algoritam za pronalaženje zapisa putem B^+-stabla

- algoritam za traženje zapisa s ključem vrijednosti **k** je rekurzivan
  - cilj je u svakom koraku rekurzije (pretraga i-te razine) pronaći čvor na nižoj, (i+1)-voj razini, koji će voditi prema listu u kojem se nalazi ključ čija je vrijednost **k**
- traženje zapisa započinje od korijena (0-te razine)
- u čvoru i-te razine potrebno je pronaći najveću vrijednost ključa koja je manja ili jednaka traženoj vrijednosti **k**
  - za prvu kazaljku internog čvora nije navedena vrijednost ključa, pa ona "pokriva" sve vrijednosti ključeva manje od vrijednosti ključa K_2
- nakon pronalaženja odgovarajuće vrijednosti ključa, slijedi se pripadna kazaljka i time se obavlja pozicioniranje na (i+1)-vu razinu
- postupak se ponavlja rekurzivno sve dok se ne dođe do lista. U njemu se mora nalaziti, ako postoji, ključ čija je vrijednost **k**, te pripadna kazaljka prema traženom zapisu (n-torki)

Dohvat podataka iz intervala, sortiranje

```
SELECT * FROM osoba
  WHERE mbr BETWEEN 28 AND 60
  ORDER BY mbr;
```



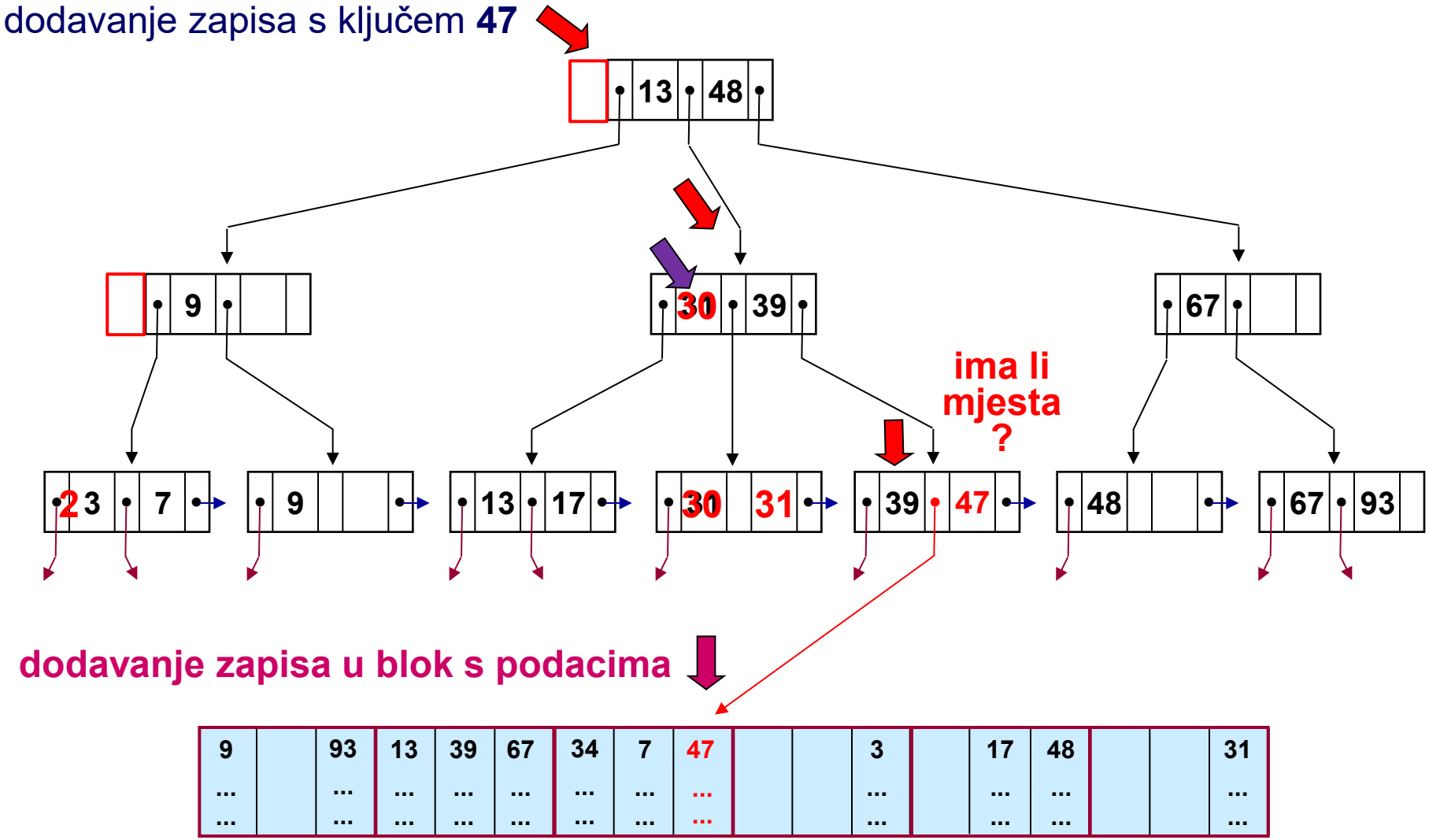
na ovoj slici nisu prikazani blokovi s podacima

- pronaći list u kojem bi trebao biti zapis s ključem **28**
- redom dohvaćati zapise iz listova dok se ne dođe do vrijednosti ključa koja je veća ili jednaka vrijednosti **60.** Za svaku dohvaćenu kazaljku dohvatiti n-torku
  - taj postupak omogućuju kazaljke među listovima
- dobiveni su svi traženi zapisi, pri tome su poredani prema mbr
- Ako se obavlja SELECT * ... ORDER BY mbr **DESC**
  - pronađene zapise jednostavno ispisati obrnutim redoslijedom

Algoritam za dodavanje zapisa u B^+-stablo

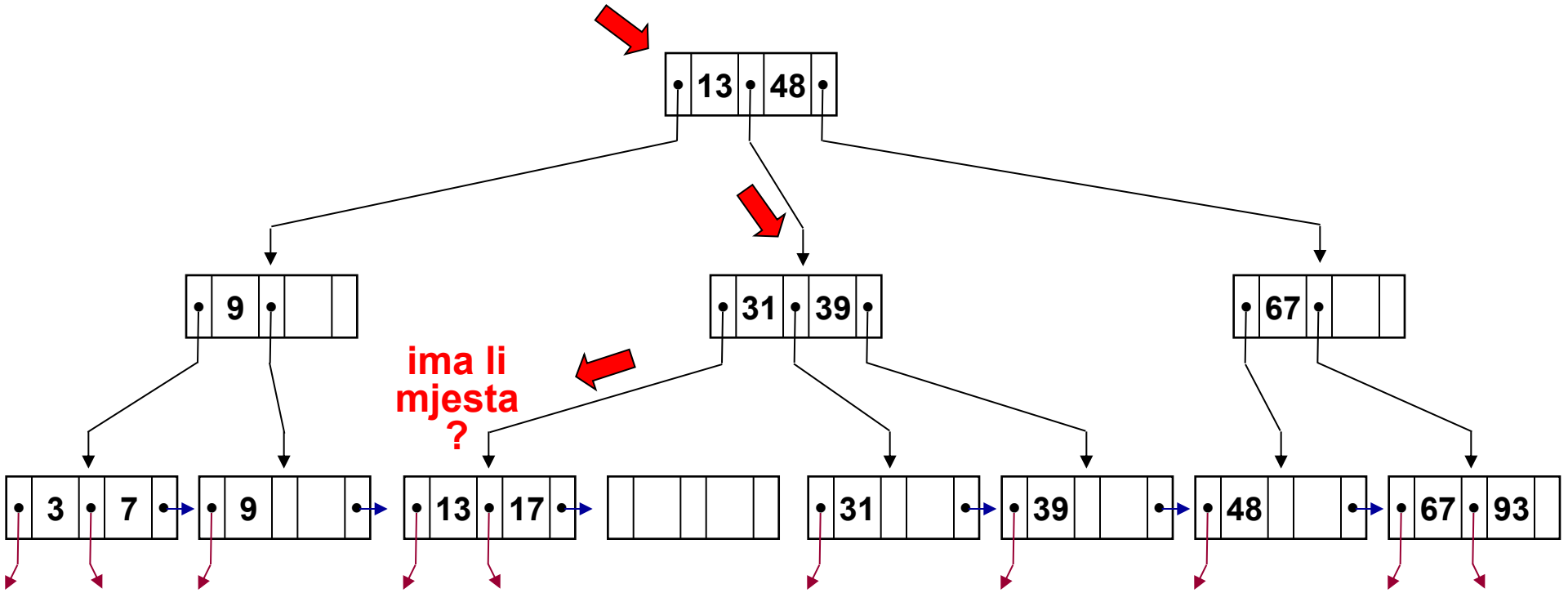
- zapis (podaci) se upisuje u jedan od slobodnih blokova s podacima
- obavlja se procedura za pronalaženje lista L u koji pripada vrijednost ključa k
- ako čvor L nije popunjen, u čvor se dodaje zapis s ključem k i kazaljkom na pripadni zapis u datoteci, uz očuvanje poretka vrijednosti ključeva
  - nova vrijednost nikad nije prva u čvoru, osim u slučaju krajnjeg lijevog čvora
- ako je čvor L popunjen
  - stvara se novi čvor i zapisi među njima se podijele, pri čemu svakom od čvorova pripadne polovica zapisa
  - budući da je dodan novi čvor, u nadređeni čvor potrebno je dodati zapis s kazaljkom i najmanjom vrijednošću ključa u novom čvoru
  - za dodavanje novog zapisa u nadređeni čvor koristi se ista procedura kao za dodavanje zapisa u čvor na nižoj razini
- postupak je rekurzivan i mora se obaviti za svaku nadređenu razinu (sve dok se ne dođe do korijena ili se na nekoj od razina nađe dovoljno mjesta za upis vrijednosti ključa i kazaljke, bez dodavanja novih čvorova).
- ako se dođe do korijena, može se desiti da u korijenu nema mjesta za novi zapis. Tada se dodaje novi čvor i formira se novi korijen na višoj razini

Dodavanje zapisa u B$^+$-stablo



Dodavanje zapisa u B$^+$-stablo

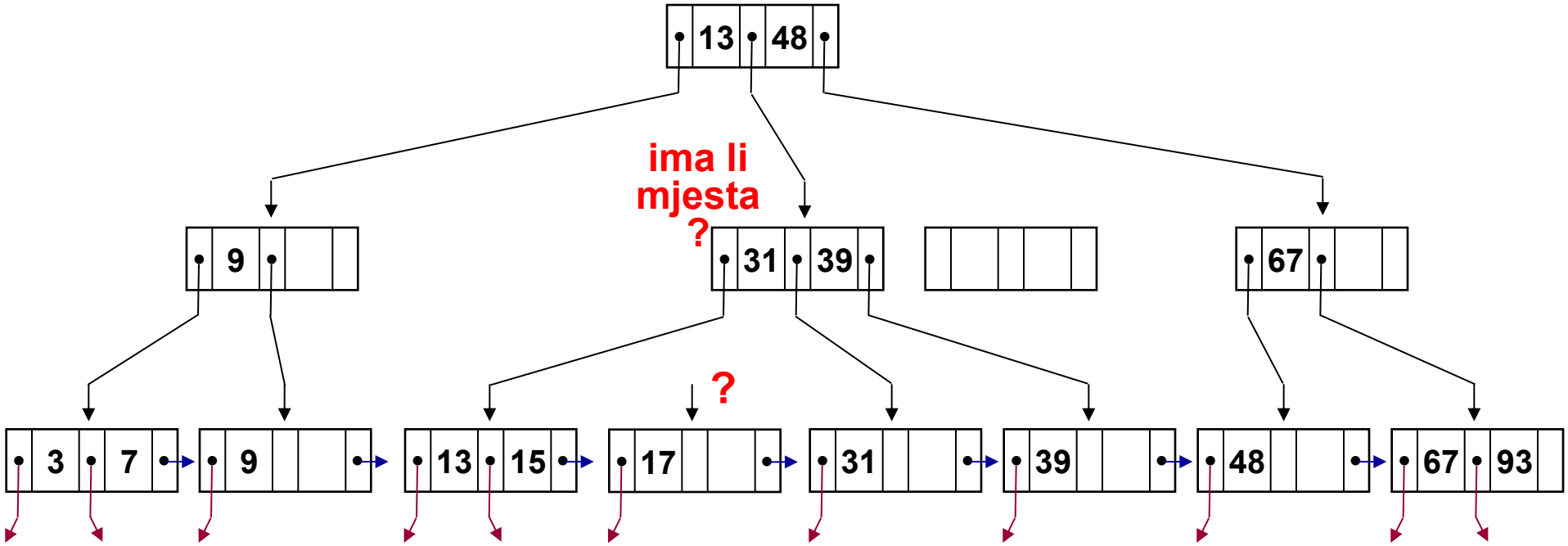
dodavanje zapisa s ključem **15**



dodati novi čvor i podijeliti zapise 13, 15, 17 među čvorovima

Dodavanje zapisa u B$^+$-stablo

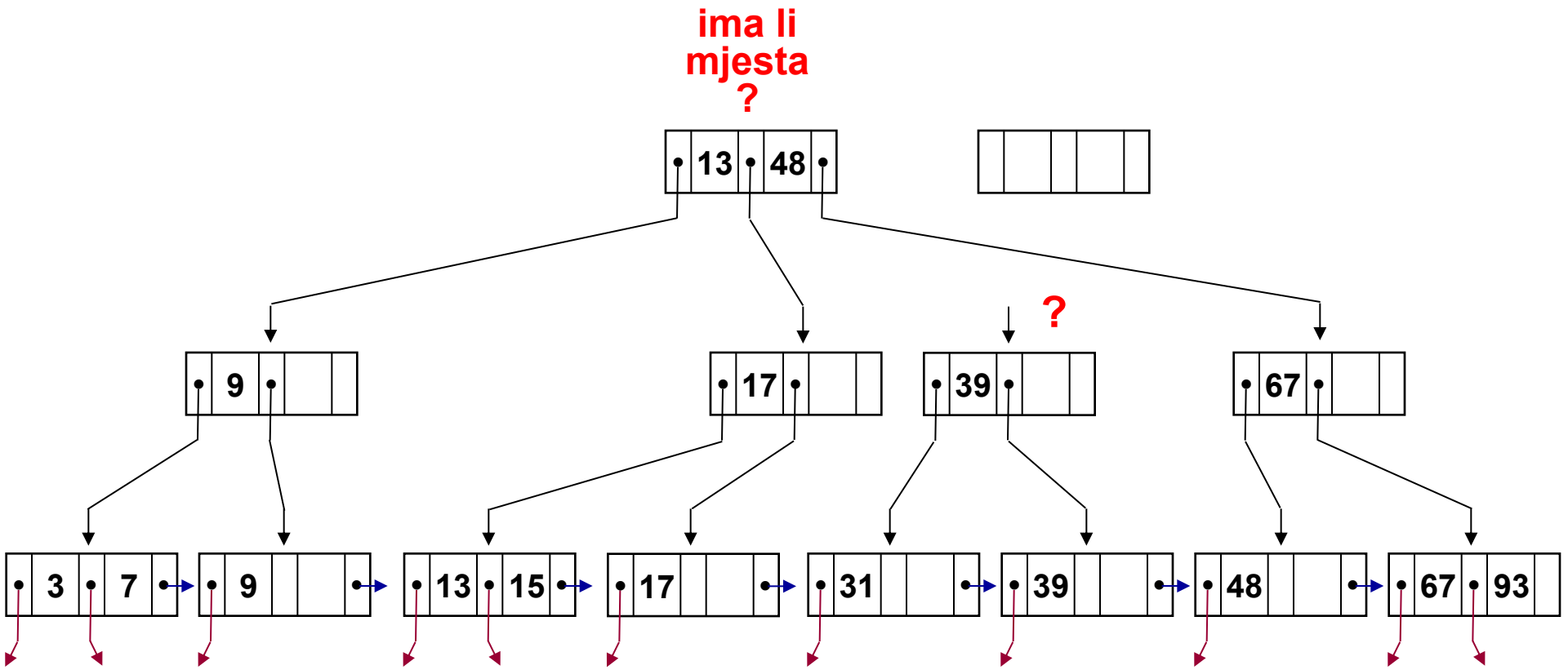
dodavanje zapisa s ključem **15**



dodati novi čvor i podijeliti zapise 13, 17, 31, 39 među čvorovima

Dodavanje zapisa u B$^+$-stablo

dodavanje zapisa s ključem **15**

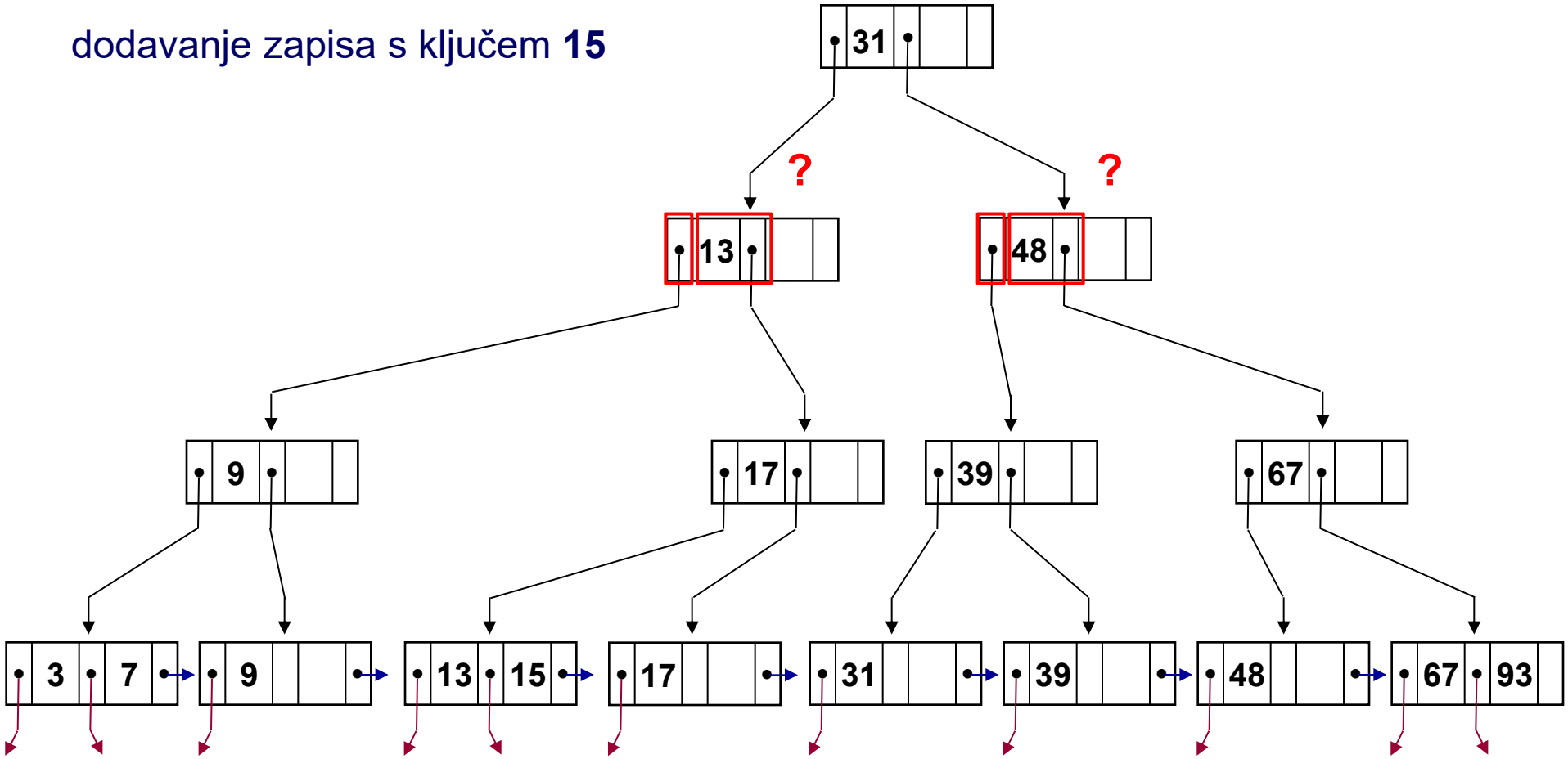


dodati novi čvor i podijeliti zapise

-∞, 13, 31, 48 među čvorovima

Dodavanje zapisa u B$^+$-stablo

dodavanje zapisa s ključem **15**



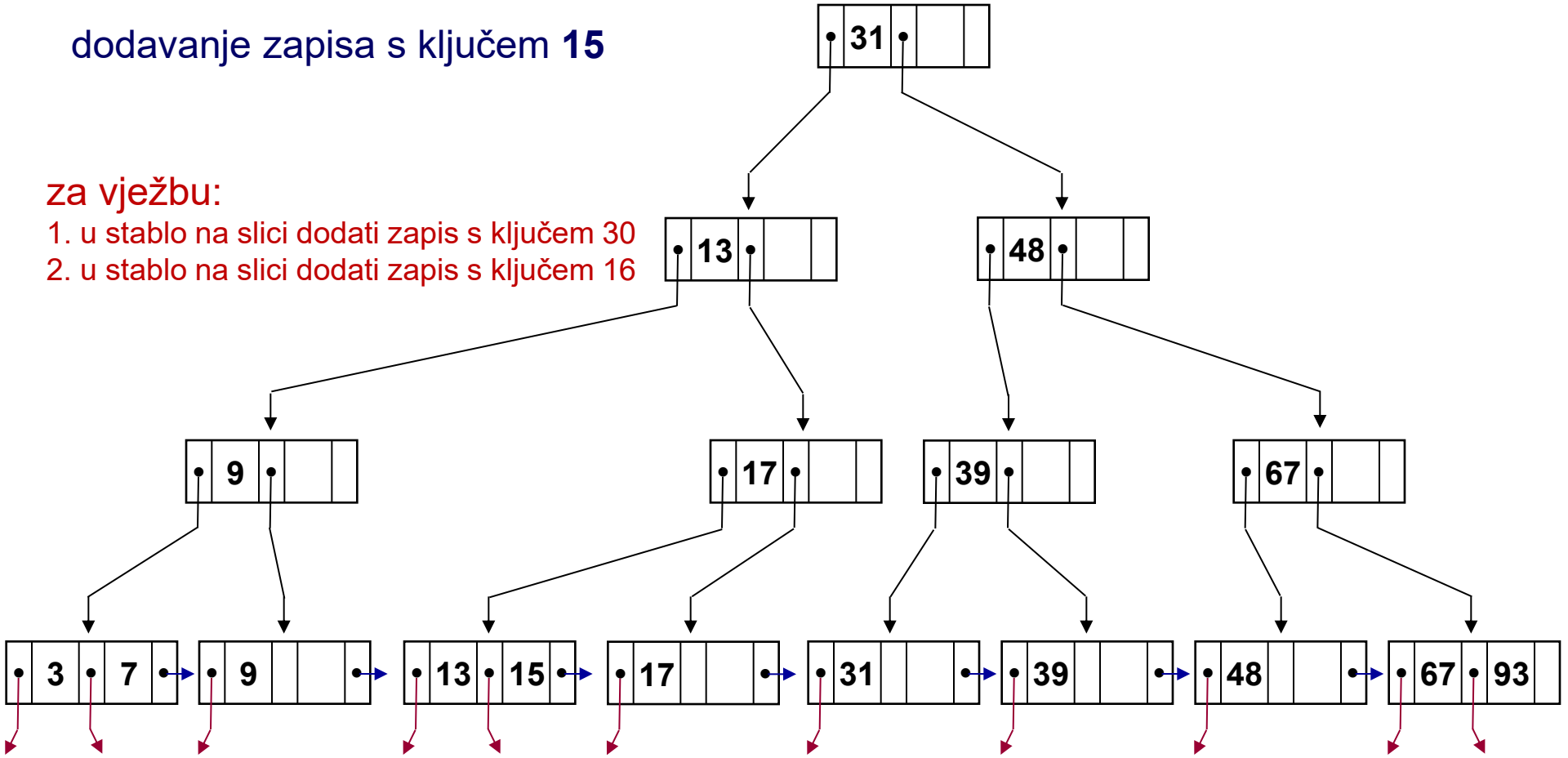
dodati novi korijen i upisati zapise -∞, 31

Dodavanje zapisa u B$^+$-stablo

dodavanje zapisa s ključem **15**

za vježbu:

1. u stablo na slici dodati zapis s ključem 30
2. u stablo na slici dodati zapis s ključem 16



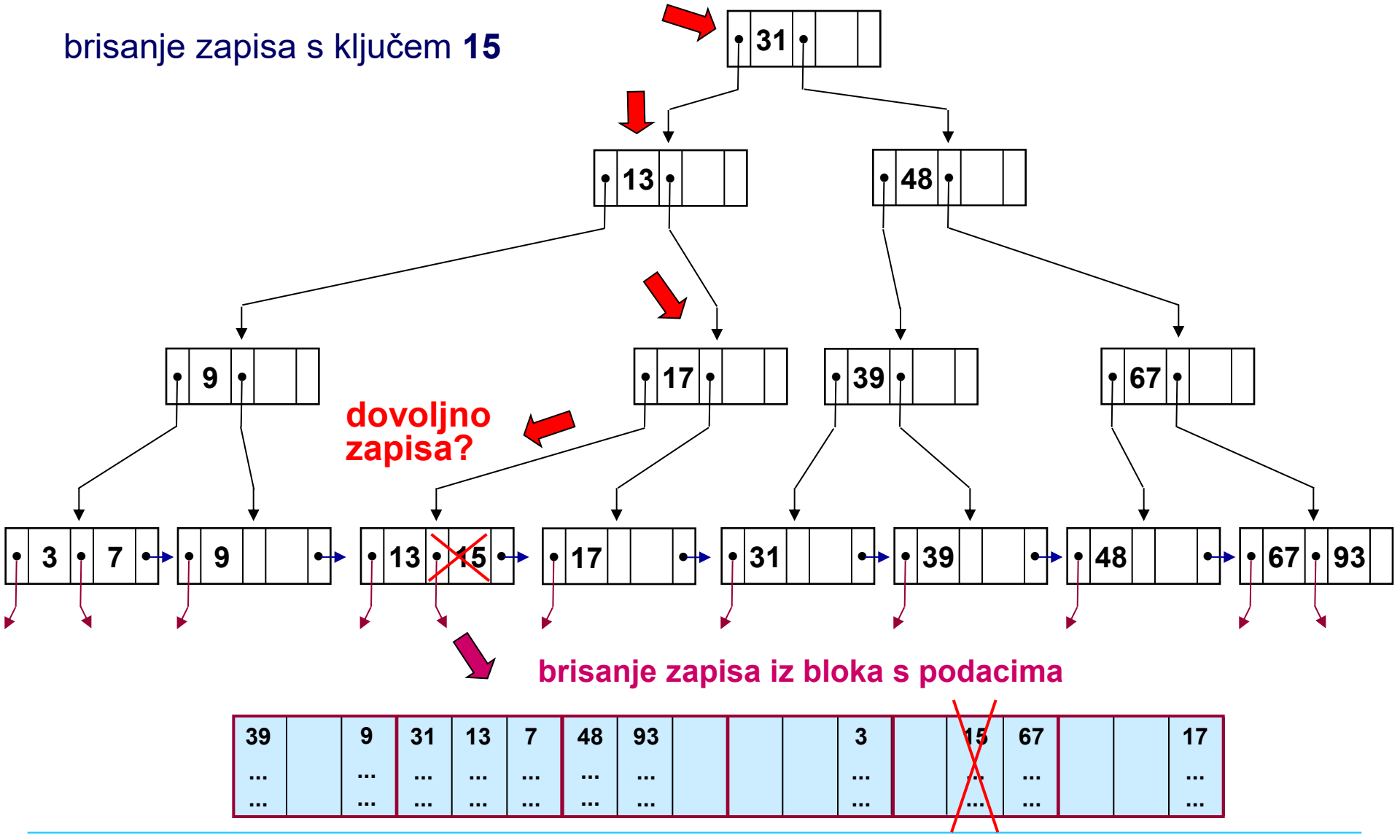
rezultat je balansirano stablo veće dubine

Algoritam za brisanje zapisa iz B^+-stabla

- obavlja se procedura za pronalaženje lista L u kojem se nalazi ključ k
- zapis se briše iz bloka s podacima, a vrijednost ključa i kazaljka iz čvora L
- ako čvor L nakon brisanja sadrži dovoljan broj zapisa
  - samo ako je potrebno (onda kada se iz lista obriše prvi zapis), trebat će promijeniti vrijednost ključa u nekom od predaka
- inače (tj. ako čvor L nakon brisanja nema dovoljan broj zapisa)
  - ako postoji čvor-brat L_X koji se nalazi neposredno s lijeva ili s desna čvoru L i ima za barem jedan više od dovoljnog broja zapisa
    - zapise podijeliti između čvorova L i odabranog L_X. Po potrebi obaviti korekciju ključa u nekom od predaka
  - inače odabrati jednog od braće koji se nalazi neposredno s lijeva ili s desna čvoru L (taj sigurno ima najmanji dopušteni broj zapisa)
    - čvorovi L i L_X se spajaju u jedan čvor. U nadređenom čvoru briše se jedan zapis te po potrebi mijenja ključ u nekom od predaka
    - brisanje zapisa u nadređenom čvoru svodi se na rekurzivno izvođenje procedure za brisanje. Ako se putem prema korijenu dođe u situaciju da treba spojiti jedina dva čvora-djeteta korijena, tada se oni spajaju, postaju novi korijen stabla, a stari se korijen briše

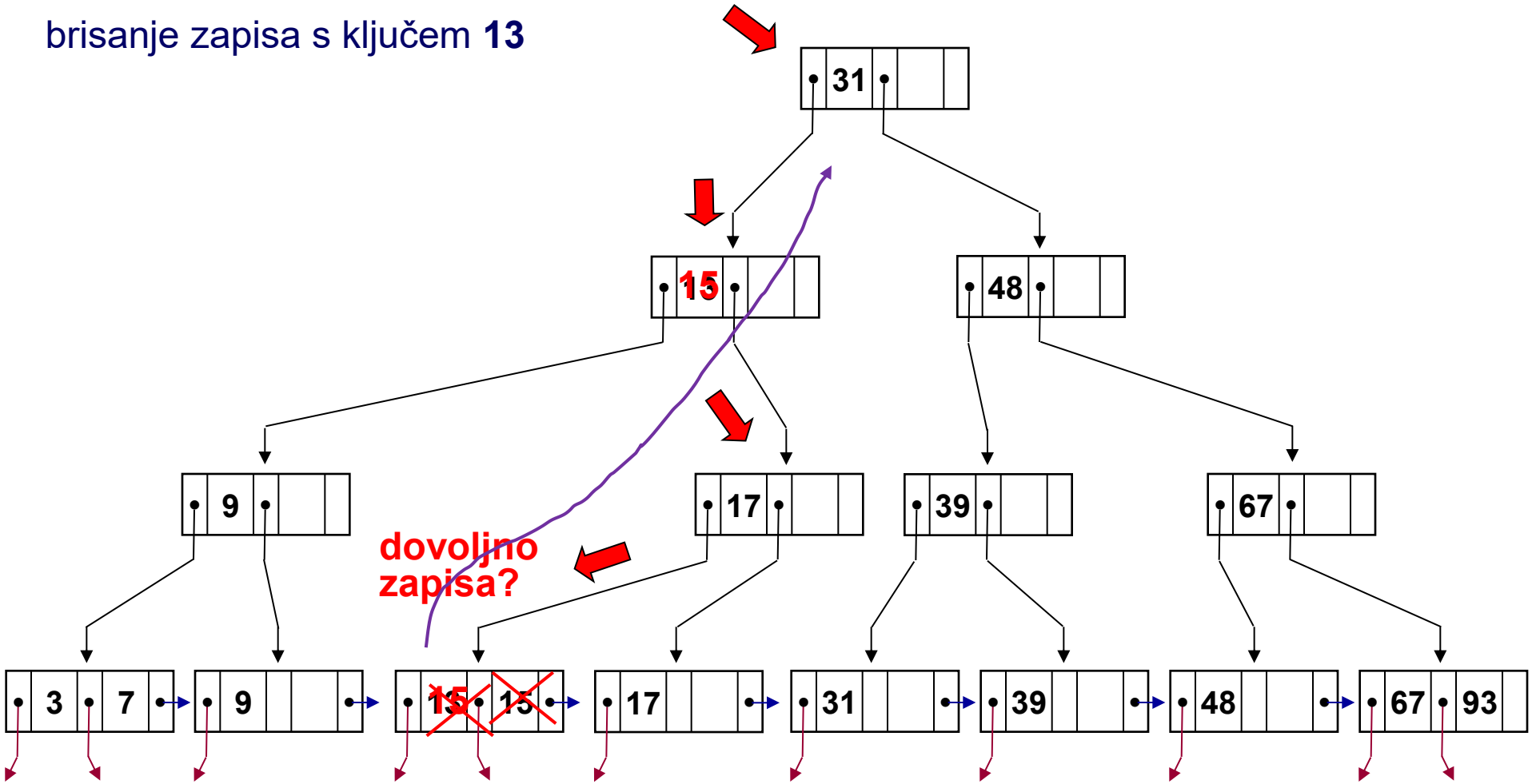
Brisanje zapisa iz B$^+$-stabla

brisanje zapisa s ključem **15**



Brisanje zapisa iz B$^+$-stabla

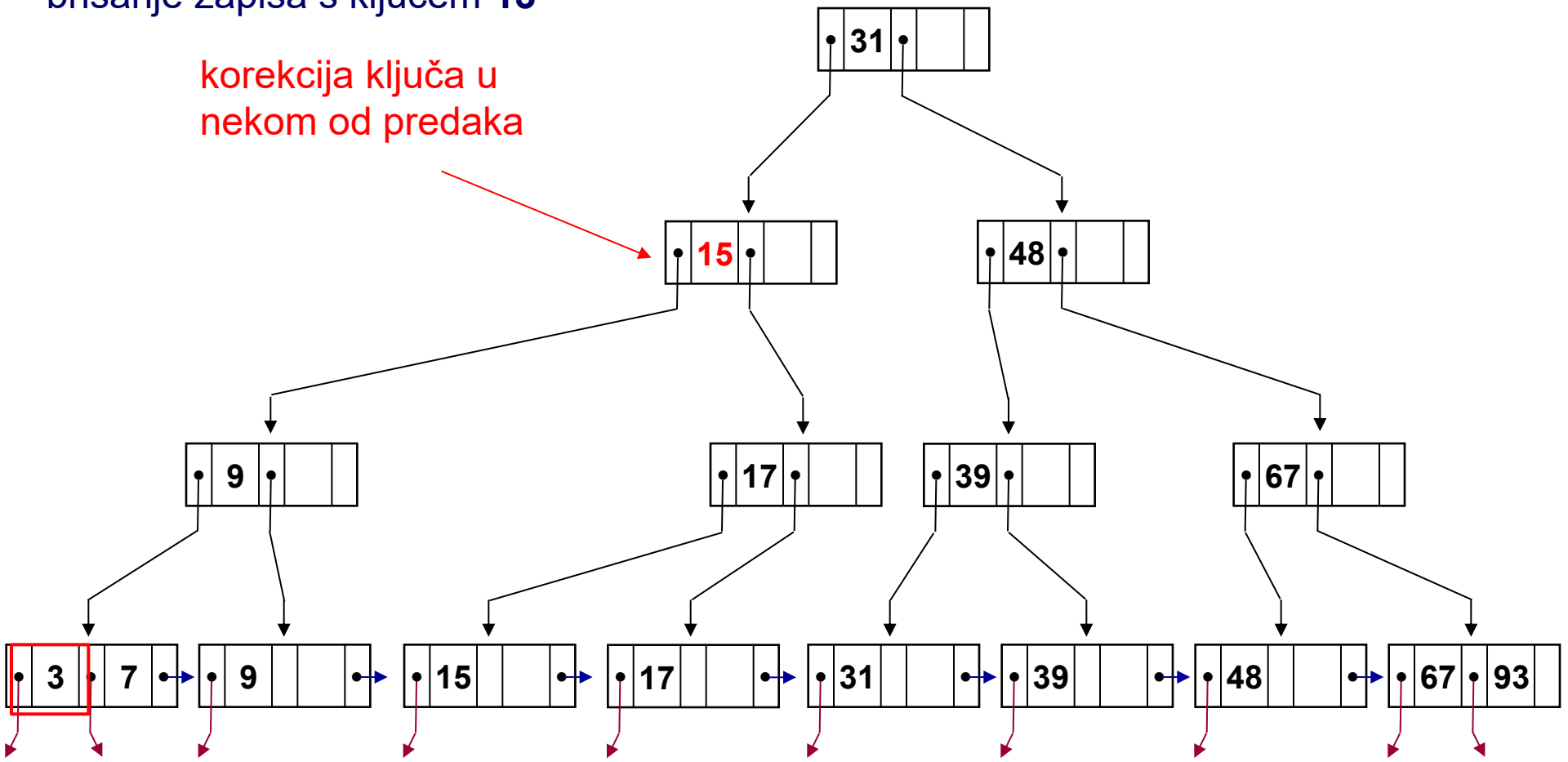
brisanje zapisa s ključem **13**



obaviti korekciju ključa u nekom od predaka
(jer sada je prvi zapis u listu 15, umjesto 13)

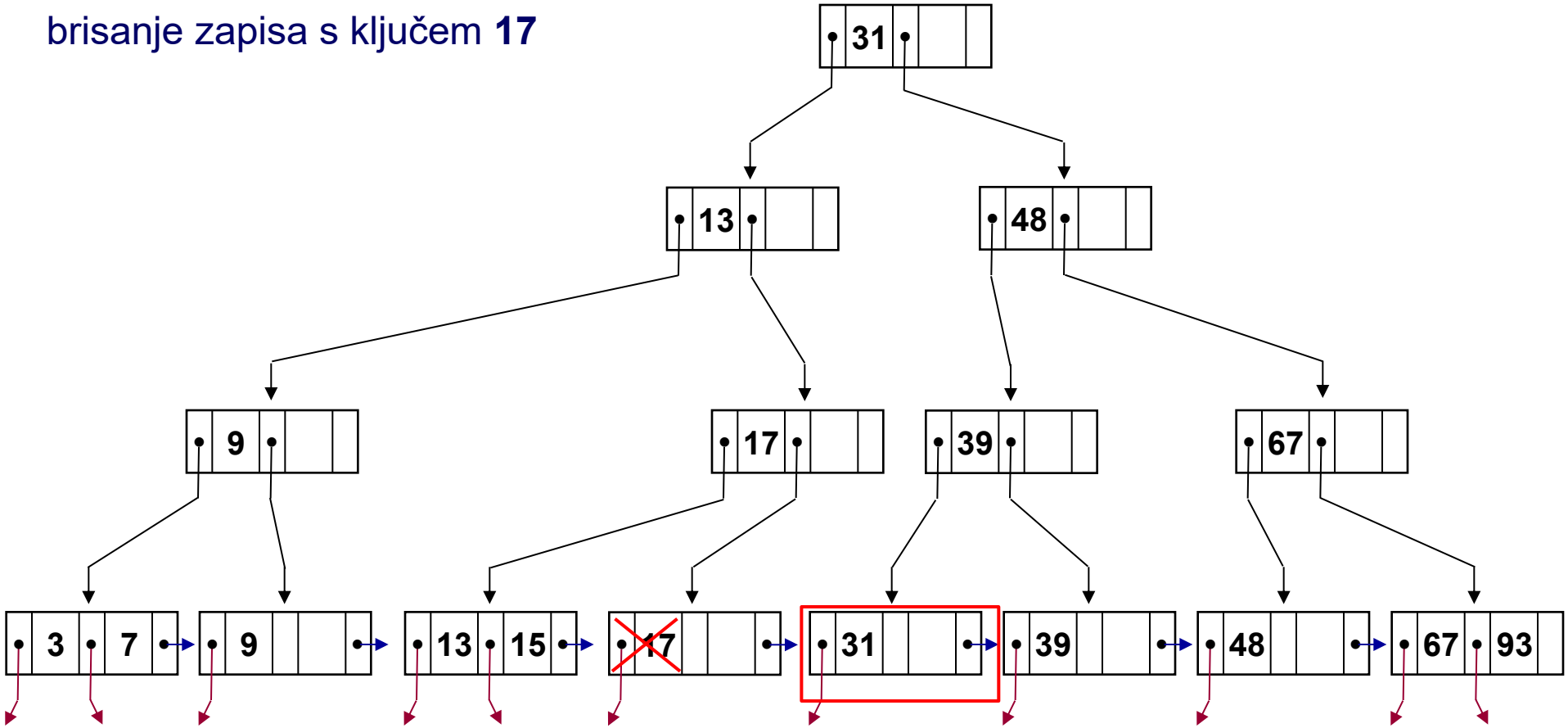
Brisanje zapisa iz B$^+$-stabla

brisanje zapisa s ključem **13**



Brisanje zapisa iz B$^+$-stabla

brisanje zapisa s ključem **17**



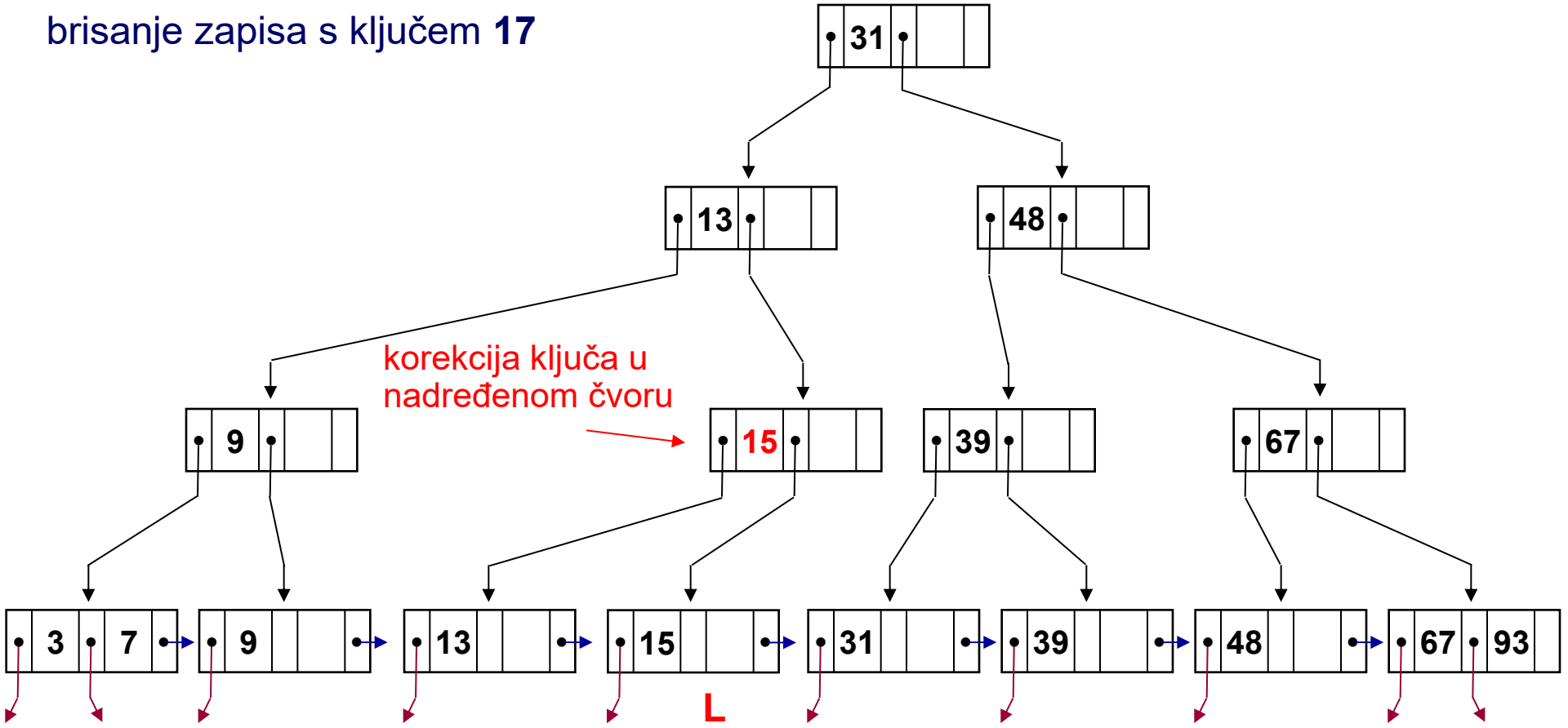
ima li brat L_X više nego treba?

ima li L dovoljno zapisa?

podijeliti zapise s bratom L_X i obaviti korekciju ključa u nekom od predaka

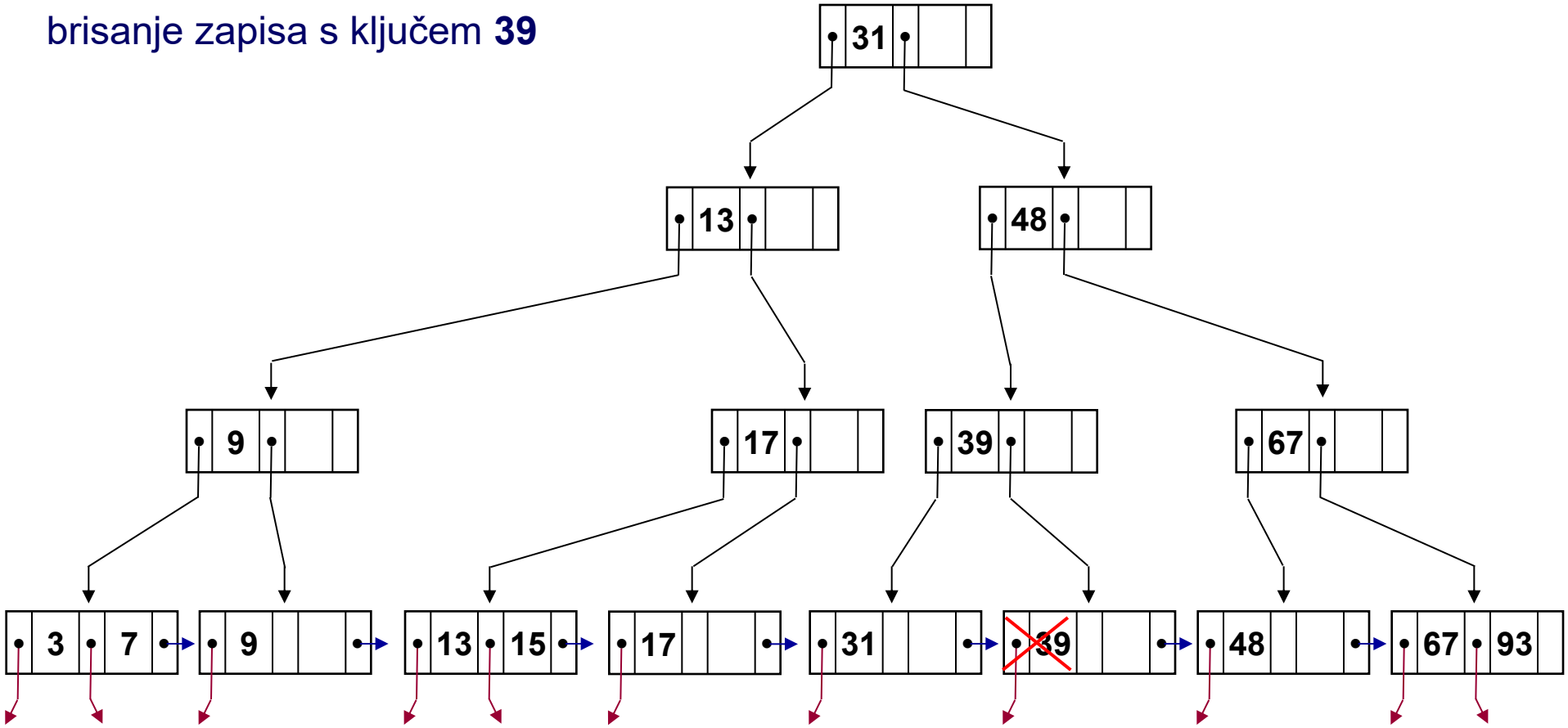
Brisanje zapisa iz B$^+$-stabla

brisanje zapisa s ključem **17**



Brisanje zapisa iz B$^+$-stabla

brisanje zapisa s ključem **39**



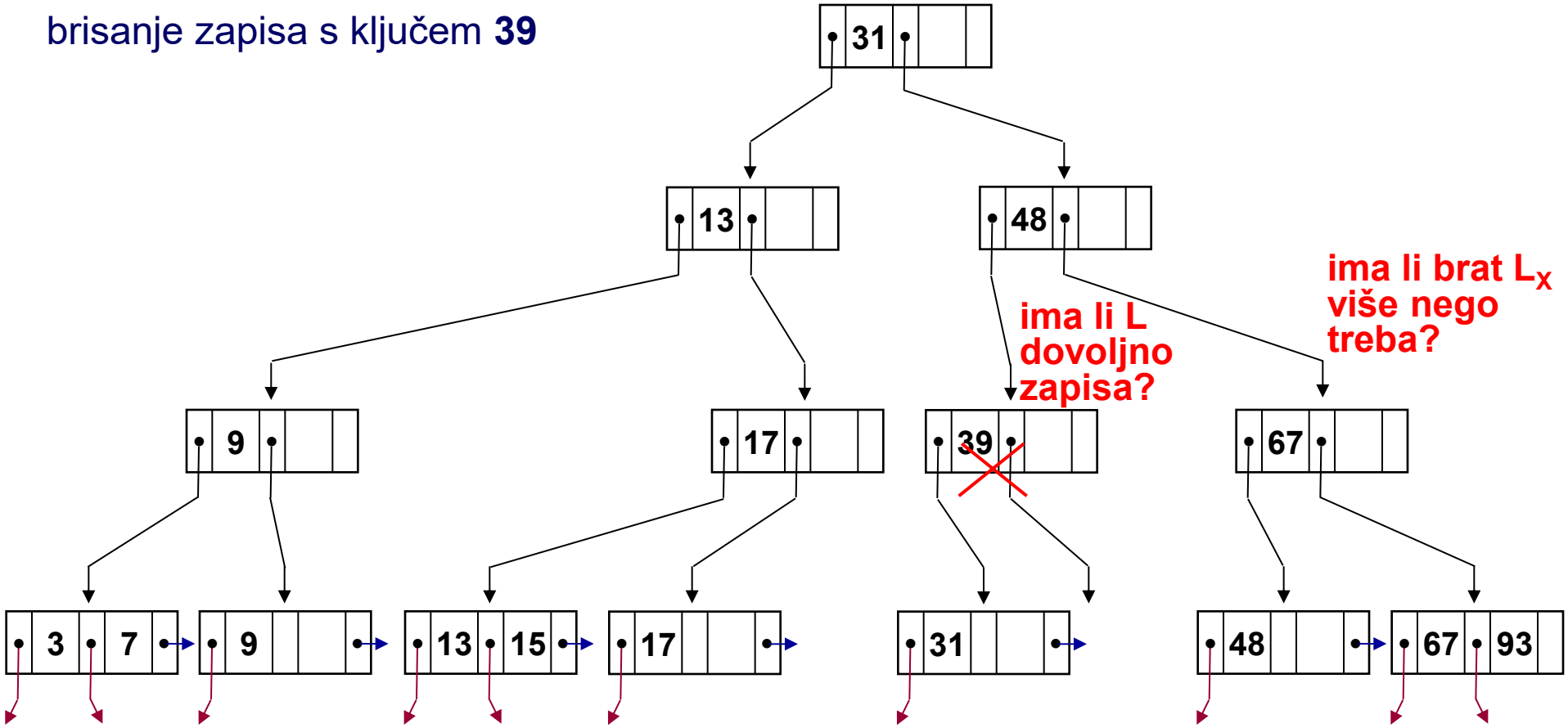
spojiti L i L_X . Obrisati zapis u nadređenom čvoru i po potrebi promijeniti vrijednost ključa u nekom od predaka.

ima li brat L_X više nego treba?

ima li L dovoljno zapisa?

Brisanje zapisa iz B$^+$-stabla

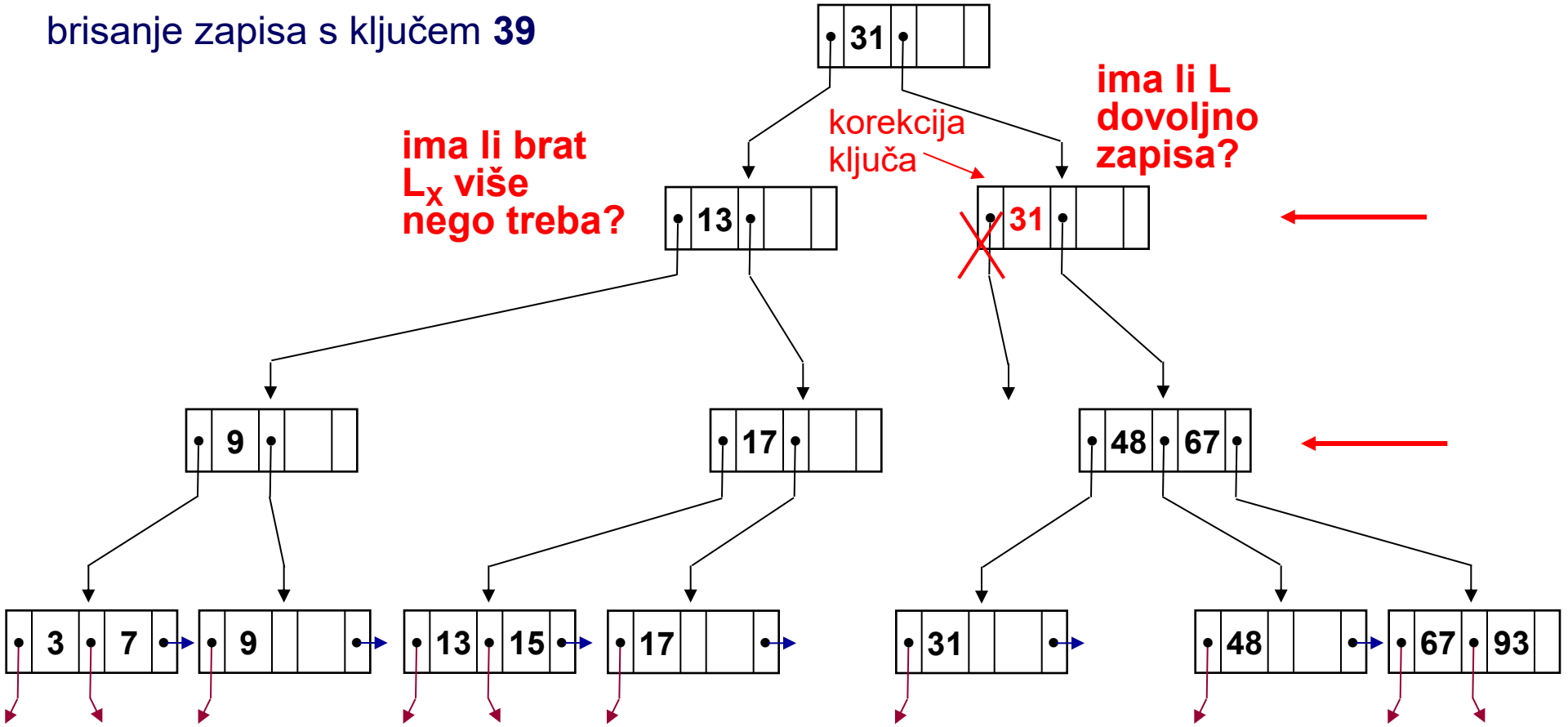
brisanje zapisa s ključem **39**



spojiti L i L_X . Obrisati zapis iz nadređenog čvora i po potrebi promijeniti vrijednost ključa u nekom od predaka.

Brisanje zapisa iz B$^+$-stabla

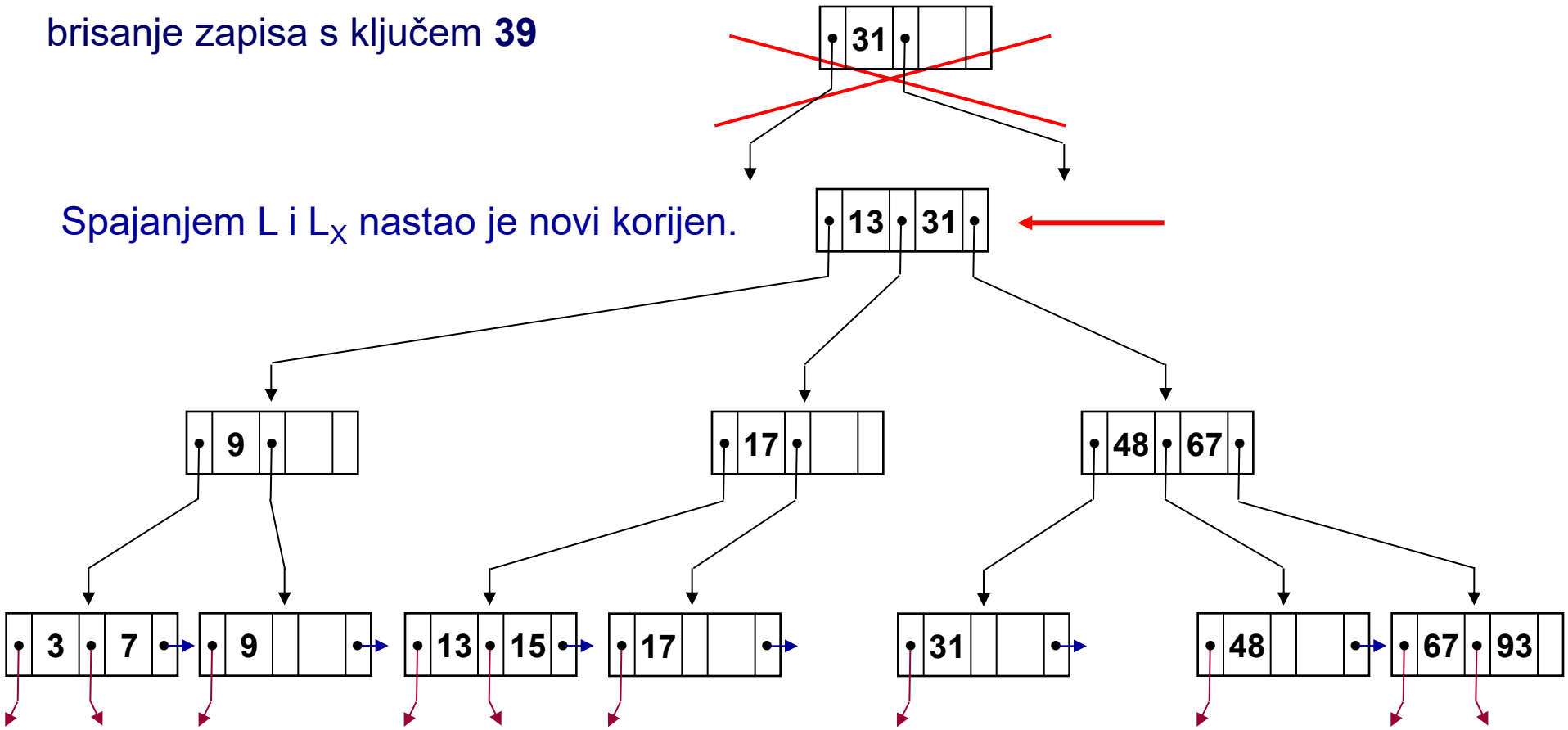
brisanje zapisa s ključem **39**



spojiti L i L_X.

Brisanje zapisa iz B$^+$-stabla

brisanje zapisa s ključem **39**



Spajanjem L i L_X nastao je novi korijen.

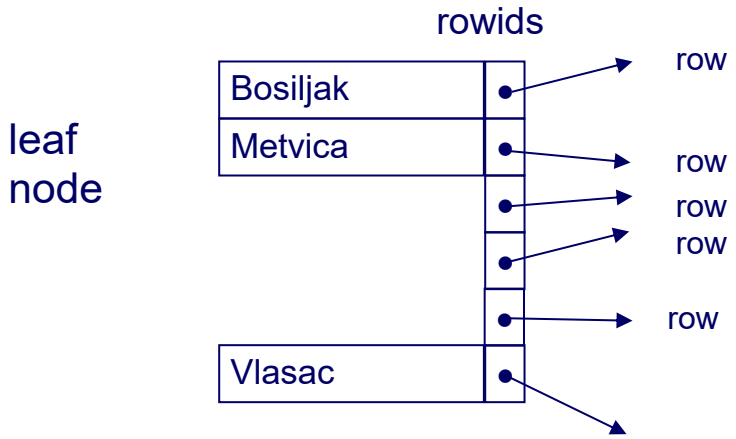
za vježbu:

1. u stablu na slici obrisati zapis s ključem 13
2. u stablu na slici obrisati zapis s ključem 48
3. u stablu na slici obrisati zapis s ključem 17 i nakon toga zapis s ključem 13

rezultat je balansirano stablo manje dubine

Zapisi podataka s jednakim vrijednostima ključa

- *non-unique index*
- opisani algoritmi za B-stablo se moraju modificirati. Problemu se pristupa na različite načine. Jedan od načina jest da se u listovima omogući pohrana više od jednog pokazivača na zapise s podacima. Npr:



Mogući problem: ako postoji vrlo veliki broj n-torki s istom vrijednošću nekog ključa

- jedno od mogućih rješenja je da se se takav zapis iz lista B-stabla poveže s jednim ili više namjenskih dopunskih blokova u koje se upišu kazaljke na blokove s podacima s tom vrijednošću ključa

Efikasnost operacije pretrage u B^+-stablu

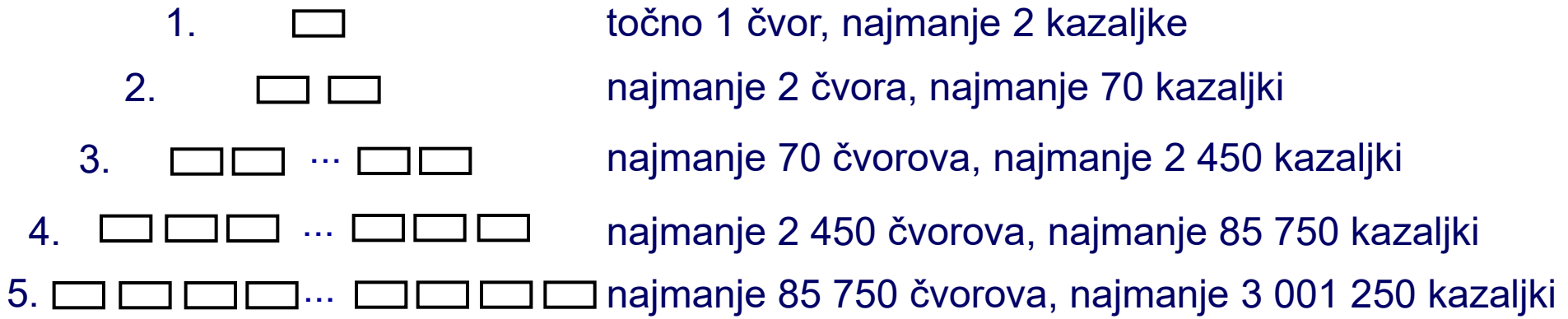
- pretpostavka: stablo reda **n** sadrži kazaljke na **m** zapisa podataka
- **n** se odabire tako da se sadržaj čvora može smjestiti u jedan fizički blok

  $\Rightarrow$ za dohvat **jednog** čvora potrebna je **jedna** U/I operacija
- broj U/I operacija u stablu pri traženju zapisa ovisi o broju razina u stablu jer se pri dohvatu zapisa mora obaviti po jedna U/I operacija za svaki čvor B-stabla na putu od korijena do lista
- B-stablo ima najveći broj razina onda kada su čvorovi najmanje popunjeni

  $\Rightarrow$ moguće je odrediti koliko će U/I operacija biti potrebno obaviti u najlošijem slučaju

Efikasnost operacije pretrage u B$^+$-stablu

- **Primjer:** za broj n-torki **m** = 1 000 000, za red stabla **n** = 70, pokazat će se da je ukupni broj razina (uključujući i razinu korijena) u najlošijem slučaju jednak 4:



- B$^+$-stablo koje bi imalo ukupno 5 razina, moralo bi imati **najmanje** 3 001 250 kazaljki na zapise. To znači da B-stablo reda 70 čije kazaljke u listovima pokazuju na 1 000 000 n-torki smije imati najviše 4 razine.

⇒ Za dohvat zapisa prema vrijednosti ključa potrebno je najviše 5 U/I operacija (4 U/I operacije za dohvat lista u kojem se nalazi kazaljka na zapis + 1 U/I operacija za dohvat bloka s podacima)

- često se korijen i njegova djeca zadržavaju u međuspremniku SUBP-a

Efikasnost operacije pretrage u B^+-stablu

- kako na općeniti način izračunati dubinu stabla u najgorem slučaju
- pretpostavka: stablo reda **n** sadrži kazaljke na **m** zapisa podataka
- stablo će imati najveći broj razina ako su čvorovi najmanje popunjeni
  - najmanja popunjenost korijena je **2**
  - najmanja popunjenost internog čvora: $\lceil \mathbf{n / 2} \rceil$
  - najmanja popunjenost lista: $\lceil \mathbf{(n - 1) / 2} \rceil \approx \lceil \mathbf{n / 2} \rceil$, za dovoljno veliki **n**
- u korijenu (1. razina) ima 1 čvor i najmanje **2** kazaljke
- na 2. razini ima najmanje 2 čvora i zato najmanje $\mathbf{2 \cdot \lceil n / 2 \rceil}$ kazaljki
- u čvorovima 3. razine ima najmanje $\mathbf{2 \cdot \lceil n / 2 \rceil \cdot \lceil n / 2 \rceil}$ kazaljki
- u čvorovima i-te razine ima najmanje $\mathbf{2 \cdot \lceil n / 2 \rceil^{i-1}}$ kazaljki
- za broj zapisa u podatkovnim blokovima stabla koje ima **d** razina vrijedi:
  - $\mathbf{m \geq 2 \cdot \lceil n / 2 \rceil^{d-1}}$
- iz toga slijedi
  - $\mathbf{d \leq \log_{\lceil n/2 \rceil}(m / 2) + 1}$
- **Primjer:** za m = 1 000 000 zapisa, n = 70, ukupni broj razina (uključujući i razinu korijena) u najgorem slučaju je 4

Popunjenost B^+-stabla u stacionarnom stanju

- analitički je dokazano i simulacijama potvrđeno:
  - ako se nad B^+-stablom obavi veliki broj operacija unosa i brisanja (slučajno odabranih vrijednosti ključeva), tada će popunjenost stabla biti $\approx \ln 2 \approx 69\ \%$

- zadatak za vježbu: grubo procijeniti kolika je (u najlošijem slučaju) popunjenost B^+-stabla nakon što se u prazno stablo unese velik broj zapisa poredanih prema ključu

Efikasnost ostalih operacija u B^+-stablu

- na efikasnost operacija unosa i brisanja utječe trošak reorganizacije B-stabla
- u rijetkim slučajevima dijeljenje ili spajanje čvorova se propagira sve do korijena stabla
- u B^+-stablima koja imaju razumno velik red stabla (npr. 10 ili više) takvi su slučajevi rijetki
  - npr. za red stabla 101: koliko puta se list može podijeliti (u najgorem slučaju) ako se u stablo unese 100 zapisa
    - najgori slučaj: list je bio pun, 100 zapisa
    - 1. uneseni zapis dijeli list na dva: list od 50 i list od 51 zapisa
    - 2. - 50. uneseni zapis neće uzrokovati dijeljenje lista
    - 51. zapis uzrokuje dijeljenje lista: listovi s 50 zapisa i 51 zapisom
    - 52. - 100. uneseni zapis neće uzrokovati dijeljenje lista
  - 100 unesenih zapisa uzrokovalo je samo dvije reorganizacije na razini listova (eventualno, ali vrlo rijetko, reorganizacija će biti potrebna i na višim razinama)

Efikasnost ostalih operacija u B^+-stablu

- može se zaključiti da će se za operacije unosa ili brisanja najčešće koristiti samo dvije U/I operacije više nego kod pretrage
  - **pretraga (što treba dodatno obaviti nakon pronalaska lista):**
    - *lookup* u *heap* datoteku - 1 U/I operacija
  - **unos ili brisanje (što treba dodatno obaviti nakon pronalaska lista):**
    - *lookup* u *heap* datoteku (1 U/I operacija)
    - *output data block* (1 U/I operacija)
    - *output leaf block* (1 U/I operacija)

- originalni algoritam za B^+-stablo pri brisanju obavlja reorganizaciju uvijek kada broj zapisa u čvoru padne ispod polovice kapaciteta čvora
  - strategija *merge-at-half*
- većina današnjih sustava reorganizaciju pri brisanju provodi tek onda kada u čvoru ne preostane niti jedan zapis
  - strategija *merge-at-empty*
  - značajno niža frekvencija reorganizacija uz neznatno lošije iskorištenje prostora

Faktor popunjenosti indeksa

- većina sustava omogućuje upravljanje razinom popunjenosti B-stabla u trenutku kada se indeks kreira ili rekreira (relacije je već popunjena)
- sintaksa može biti različita. Prikazani primjer odnosi se na sustav SQL Server (Microsoft). Zadaje se postotak popunjenosti listova

```
CREATE INDEX studIdx1 ON student(jmbag)
   WITH FILLFACTOR = 95; -- svaki list je inicijalno popunjen 95%

CREATE INDEX ispitIdx1 ON ispit(dat_ispit)
   WITH FILLFACTOR = 25; -- očekuje se upis mnogo dodatnih n-torki
CREATE INDEX ispitIdx2 ON ispit(sif_predmet)
   WITH FILLFACTOR = 25;
```

- utječe na
  - inicijalnu veličinu prostora za pohranu B-stabla
  - broj U/I operacija pri dohvatu iz B-stabla (zbog veće/manje dubine stabla)
  - broj U/I operacija pri upisu u B-stablo (zbog veće/manje vjerojatnosti potrebe za reorganizacijom stable pri unosu novih zapisa)

Trošak korištenja indeksa

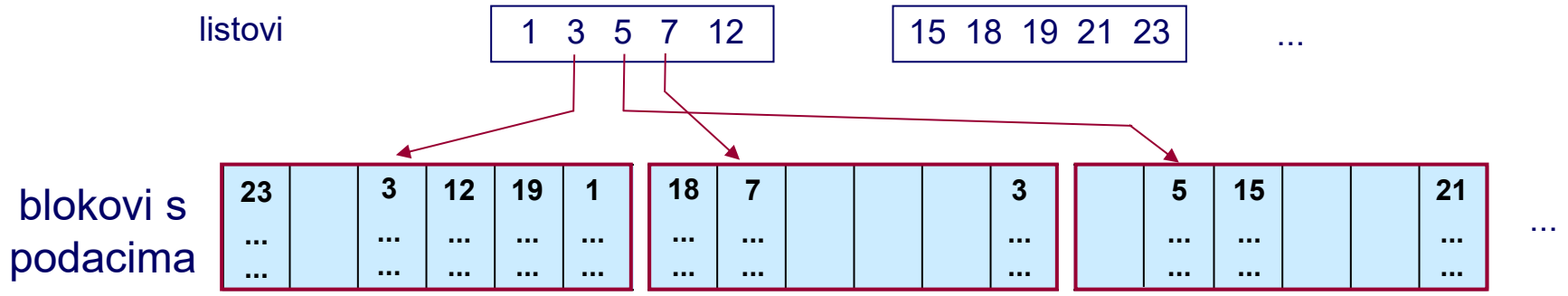
- Izgradnja i održavanje indeksa može biti skupo (prostor za pohranu, CPU, U/I aktivnosti). Administrator sustava treba osigurati da dobici pri korištenju indeksa nadmašuju gubitke koji nastaju zbog održavanja indeksa

- gruba procjena: obavljanje DML operacije INSERT ili DELETE zbog koje je potrebno obaviti održavanje jednog indeksa rezultira trostruko većim dodatnim utroškom resursa. Npr, INSERT ili DELETE operacija nad relacijom za koju su izgrađena tri indeksa je desetak puta skuplja (CPU, U/I) od iste operacije nad relacijom za koju nije izgrađen niti jedan indeks

- posebnost UPDATE operacije: primjer
  - relacija r(A, B), za atribut A je kreiran indeks
  - **`UPDATE r SET B=x WHERE A=y`**
    - nema troška održavanja indeksa, dobar odabir
  - **`UPDATE r SET A=x WHERE B=y`**
    - indeks ne donosi dobitak, postoji samo trošak održavanja indeksa, loš odabir

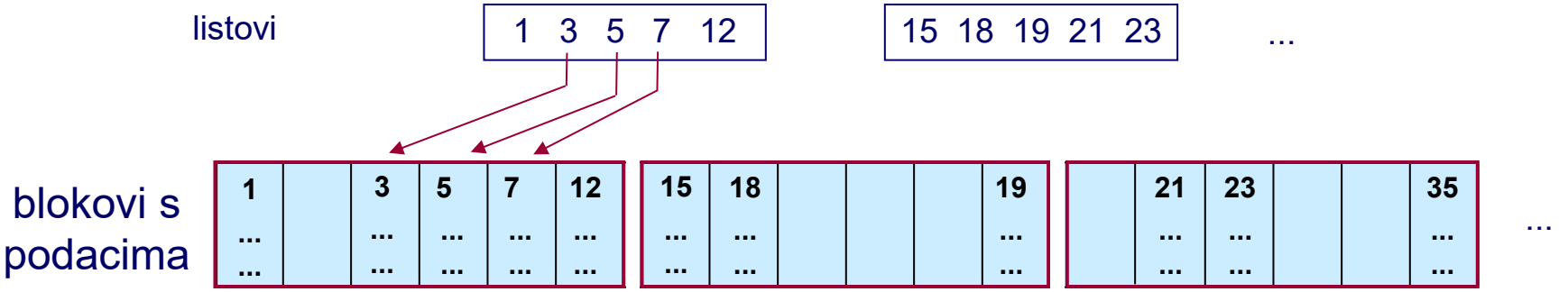
Efikasnost dohvata iz intervala vrijednosti
Indeks s poredanim zapisima podataka (*IBM Informix: Cluster index*)

- dohvat podataka u intervalu vrijednosti [3, 7]



- *Cluster index:* podaci (u podatkovnim blokovima) su poredani prema ključu

```
CREATE CLUSTER INDEX idxName ON tabname (colname, ...);
```

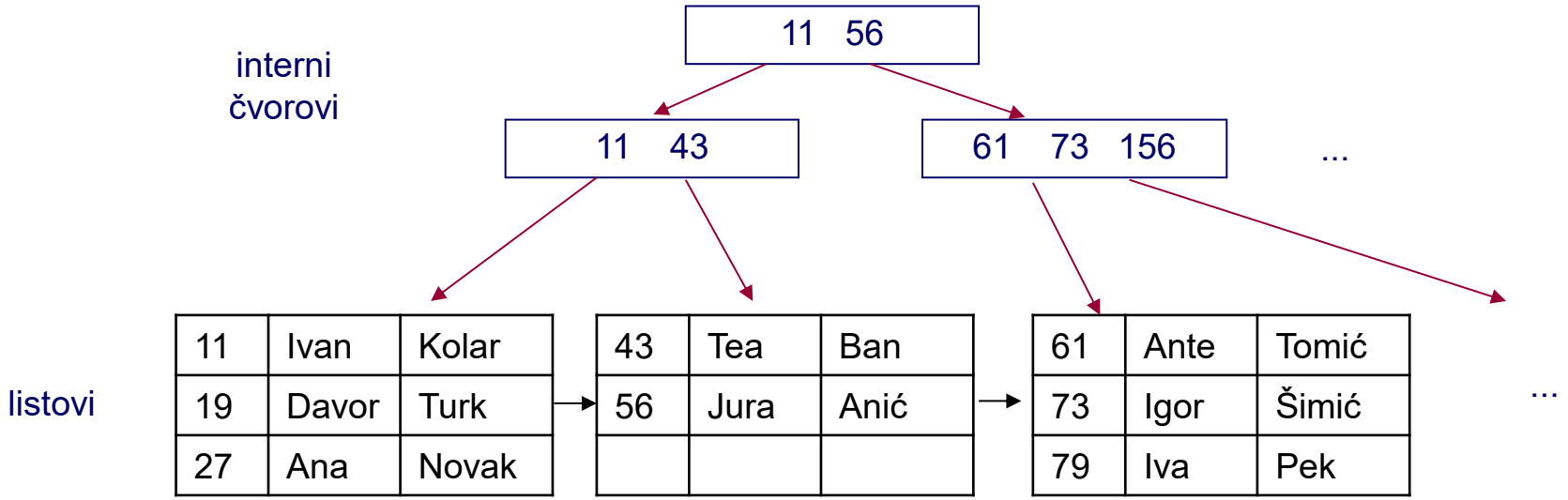


- usporediti broj U/I operacija u odnosu na non-clustered indekse
- za jednu relaciju moguć je najviše jedan indeks s poredanim zapisima podataka

Efikasnost dohvata iz intervala vrijednosti
Indeks sa zapisima podataka u listovima (*SQL Server: Clustered index*)

- naredba kreira indeksnu strukturu sličnu B⁺-stablu. Podaci (n-torke) su smješteni u listovima stabla

```
CREATE CLUSTERED INDEX idxName ON tabname (colname, ...);
```



- očito, i ovdje je moguć samo jedan *clustered index* po relaciji
- za razliku od *non-clustered* indeksa, gdje n-torka nakon promjene ključa ostaje u istom podatkovnom bloku (treba "popraviti" samo interne čvorove i listove), kod *clustered* indeksa će trebati fizički preseliti cijelu n-torku
  - npr. kod promjene matičnog broja 27 u 58

Efikasnost dohvata iz intervala vrijednosti
Indeks sa zapisima podataka u listovima (*SQL Server: Clustered index*)

- preporuča se koristiti
  - za atribute ili skupove atributa (složeni ili kompozitni indeks) prema kojima se često obavlja dohvat većeg broja n-torki iz intervala vrijednosti ili prema kojima se često obavlja sortiranje. Npr. korisno je kreirati *clustered* indeks za jmbag ako se često obavljaju upiti oblika
    - SELECT * FROM student WHERE jmbag BETWEEN 100 AND 200;
    - SELECT * FROM student ORDER BY jmbag;
- ne preporuča se koristiti
  - za atribute čije se vrijednosti često mijenjaju (to vrijedi i za ostale indekse, ali ovdje naročito)
  - naročito ne za atribute ili skupove atributa čije vrijednosti zauzimaju *puno* prostora (*wide keys*). Npr. kompozitni indeks za ime_stud + prez_stud
    - ostali indeksi kreirani nad tom relacijom, umjesto uobičajene kazaljke na blok u kojem se nalazi n-torka, moraju koristiti ključ *clustered* indeksa. Zašto: jer n-torka može promijeniti svoju fizičku poziciju (preseliti se u neki drugi list) kada se promijeni vrijednost ključa *clustered indeksa*. Stoga je drugi indeksi mogu izravno adresirati jedino preko vrijednosti ključa *clustered* indeksa i uz upotrebu tog indeksa.

Funkcijski indeks

```
CREATE TABLE put (
   id INTEGER
 , v FLOAT    -- brzina
 , t FLOAT); -- vrijeme

CREATE INDEX idxPutV ON put(v);
CREATE INDEX idxPutT ON put(t);
```

- koriste li se indeksi u upitu?

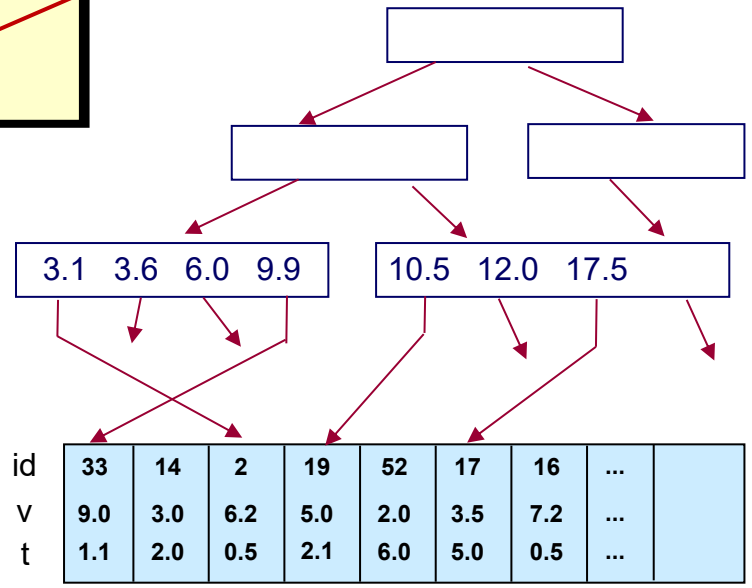
```
SELECT * FROM put
  WHERE v * t = 10.5;
```

  - NE, moguća je jedino slijedna pretraga
- umjesto dodavanja redundantnog atributa *s*, pa izgradnje indeksa nad tim atributom, bolje je koristiti funkcijski indeks:

```
CREATE FUNCTION s(brzina FLOAT, vrijeme FLOAT)
  RETURNING FLOAT WITH(NOT VARIANT)
   RETURN brzina * vrijeme;
END FUNCTION;
CREATE INDEX idxPutS ON put(s(v, t));
```

```
SELECT * FROM put
  WHERE s(v, t) = 10.5;
```

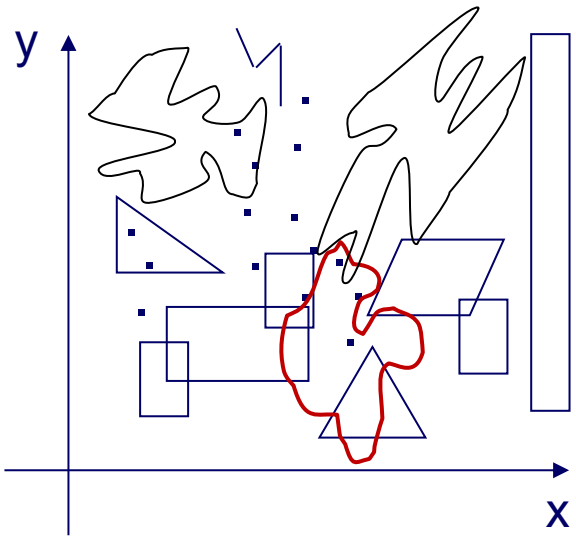
B-stablo u kojem su vrijednosti ključeva duljine prijeđenog puta (brzina x vrijeme)



- Ključevi u čvorovima B-stabla su realni brojevi koji predstavljaju izračunatu duljinu puta
- Prikazan je primjer funkcijskog izraza za sustav IBM Informix (u drugim sustavima sintaksa može biti drugačija).

Višedimenzionalni indeksi (*Multidimensional indexes*)

- U B-stablu ključ pretrage je uvijek neka skalarna vrijednost - zadana vrijednost ključa pretrage uspoređuje se s vrijednostima ključeva u zapisima B-stabla i tako se pronalaze blokovi u kojima se nalazi traženi zapis (n-torka)
- ako umjesto skalarnih vrijednosti treba indeksirati složene ili višedimenzionalne vrijednosti, indeksi temeljeni na B-stablu više neće biti dovoljni. Potrebni su tzv. višedimenzionalni indeksi

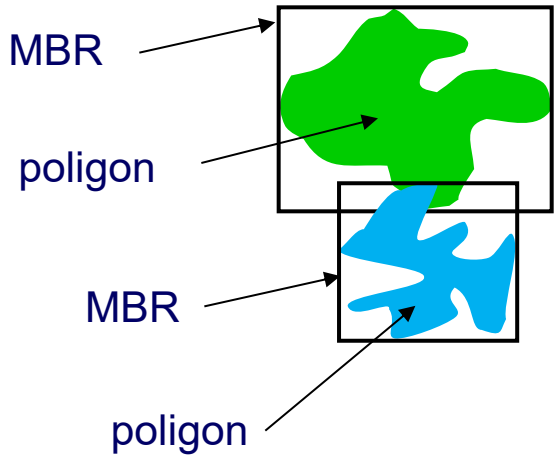


- R-stabla
- točke (*point data*), linije (*lines*), regije (*region data*)

- primjeri prostornih upita (*spatial query*)
  - dohvati sve objekte koji se nalaze na zadanoj udaljenosti od zadanog objekta
  - dohvati objekte čije se površine preklapaju sa zadanim objektom

R-stablo

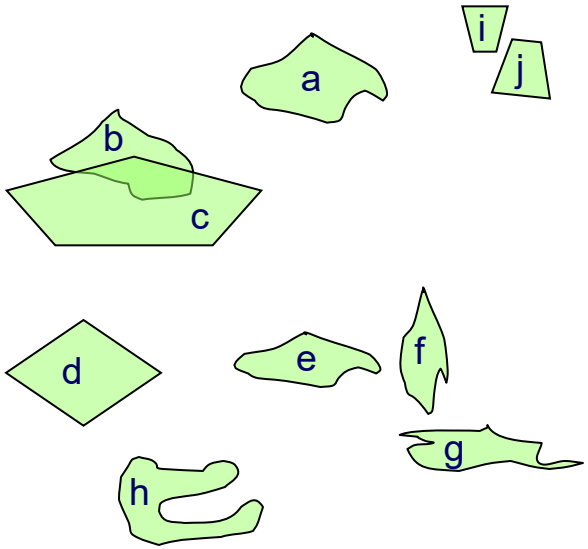
- prema osnovnoj strukturi slično je B^+-stablu
- R-stablo je balansirano stablo u kojem su ključevi uz kazaljke u stablu i ključevi pretrage minimalni granični okviri, *minimum bounding rectangle*, MBR
  - minimalni pravokutnik (2D) ili minimalni kvadar (3D) koji u cijelosti obuhvaća lik (2D) ili tijelo (3D)



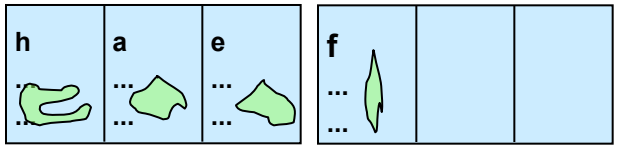
- MBR: x-low, y-low; x-high, y-high

R-stablo

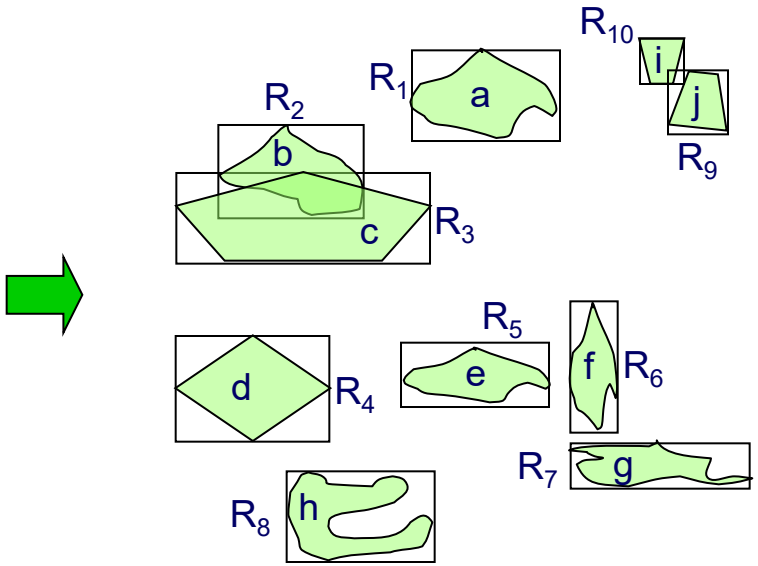
- primjer: izgraditi R-stablo reda 3 za objekte na sljedećoj slici



blokovi s podacima

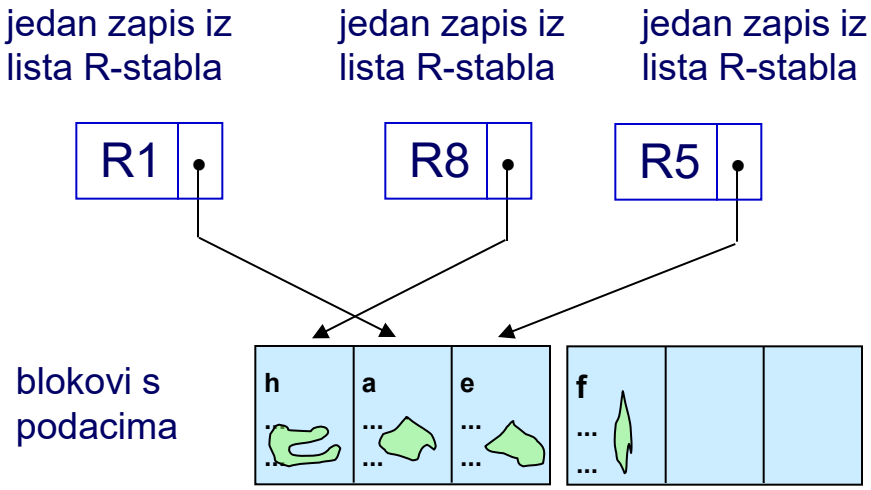


- odrediti MBR-ove za sve objekte



R-stablo

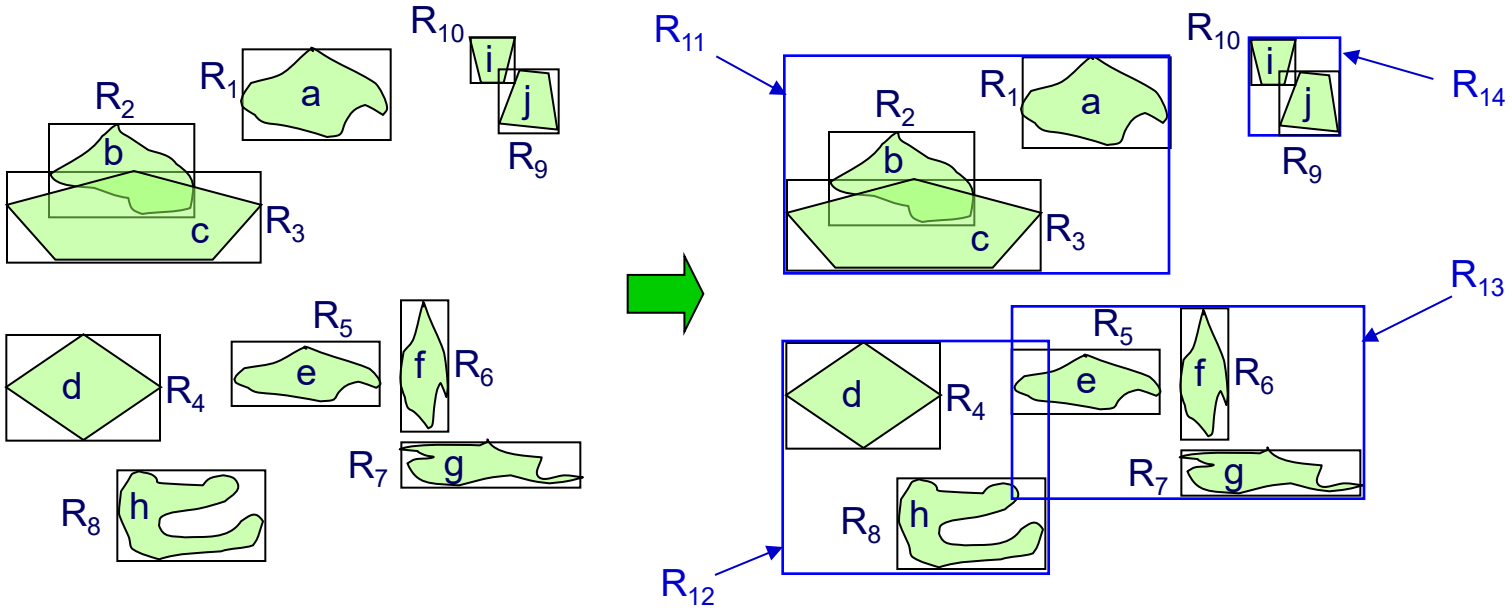
- što sadrži svaki pojedini zapis u listu R-stabla?



- jedan zapis u listu R-stabla sadrži podatke o MBR-u za objekt (x-low, y-low; x-high, y-high) i pokazivač na blok podataka (ili eventualno dodatno i poziciju zapisa u bloku) u kojem se nalaze podaci o objektu za taj MBR
  - slično kao što jedan zapis u listu B-stabla sadrži vrijednost ključa i pokazivač na blok podataka (ili eventualno dodatno i poziciju zapisa u bloku) u kojem se nalaze podaci za n-torku s dotičnim ključem

R-stablo

- primjer (nastavak):
- grupirati međusobno bliske MBR-ove u po jedan MBR koji ih prekriva. Broj MBR-ova u jednoj grupi ograničen je kapacitetom čvora, tj. redom stabla
  - R-stablo je balansirano stablo, minimalna popunjenost čvorova (osim korijena) je 50%

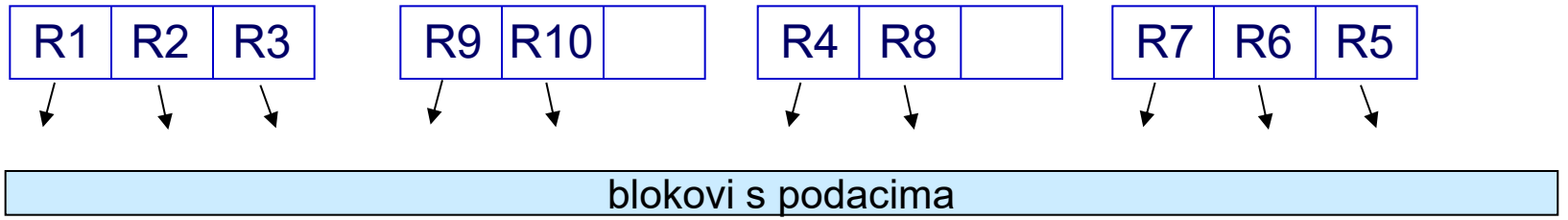


- način grupiranja nije jednoznačan, ali nastoji se minimizirati površina MBR-a

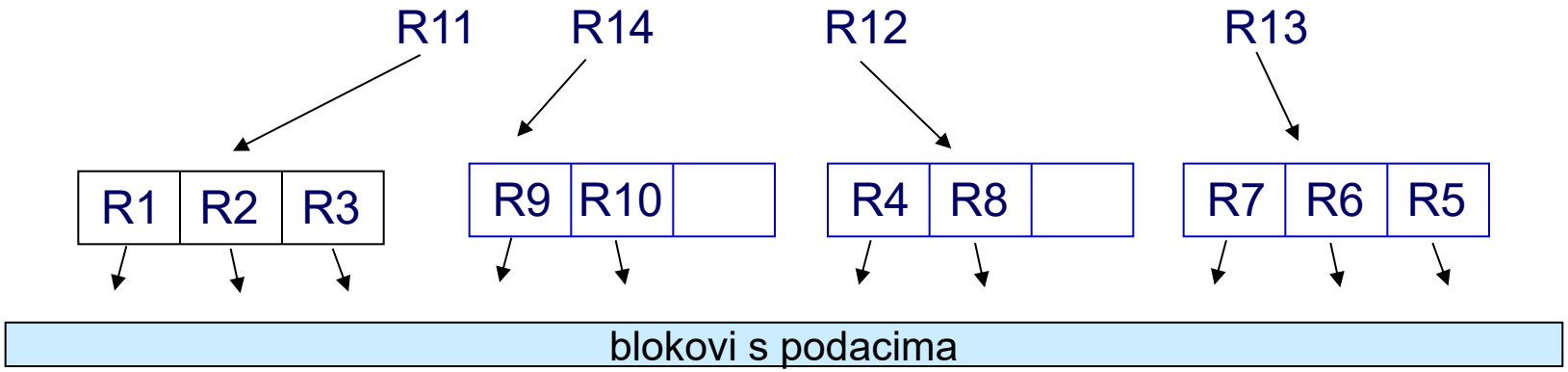
* na sljedećim slikama, radi preglednosti, objekti nisu ucrtani u MBR-ove

R-stablo

- primjer (nastavak):
- zapisi za MBR-ove jedne grupe upisuju se u jedan list

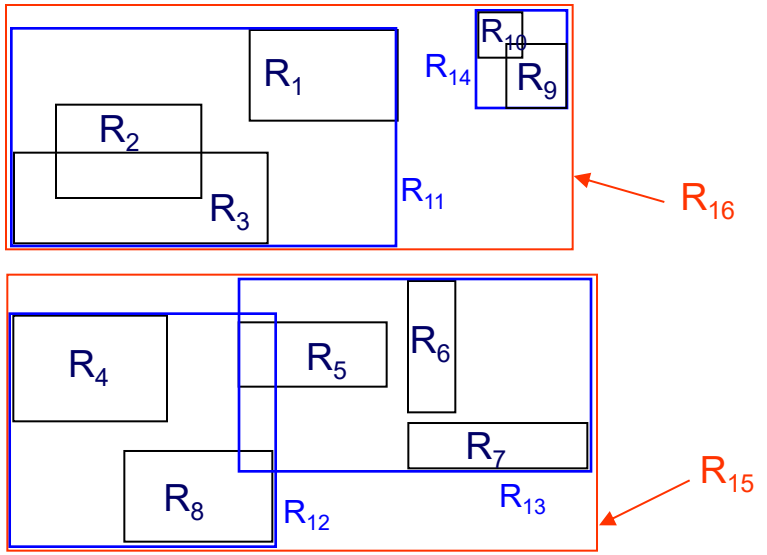


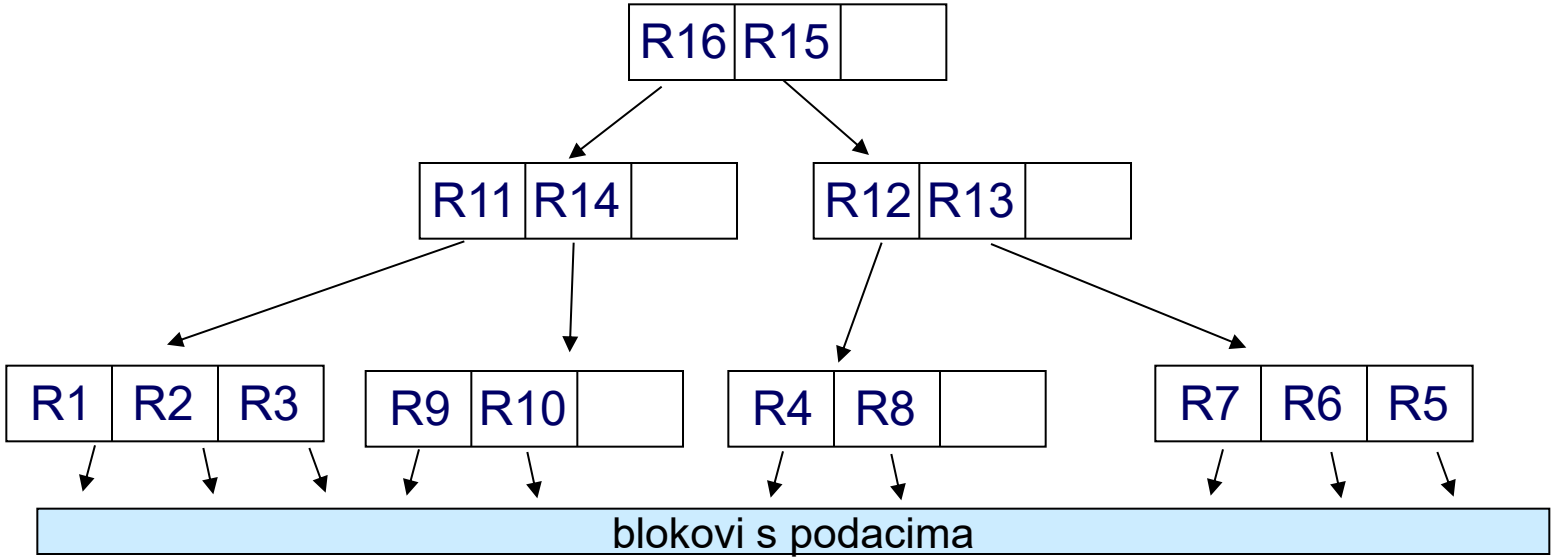
- na svaki list pokazuje po jedan zapis s više razine stabla. U zapisu se nalaze podaci o MBR-u (x-low, y-low; x-high, y-high) koji obuhvaća grupu MBR-ova iz lista i pokazivač na taj list



R-stablo

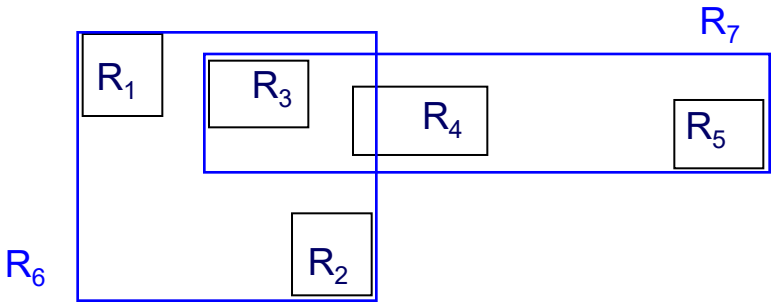
- primjer (nastavak):
- postupak se ponavlja rekurzivno. MBR-ovi s razine iznad lista grupiraju se u grupe međusobno bliskih MBR-ova, zapisi jedne grupe upisuju se u jedan interni čvor, na višoj razini se za svaku grupu formira jedan zapis (MBR + kazaljka) itd.
- postupak se ponavlja dok se ne dođe do razine korijena R-stabla



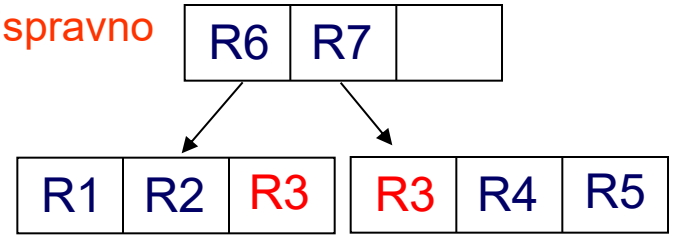


R-stablo

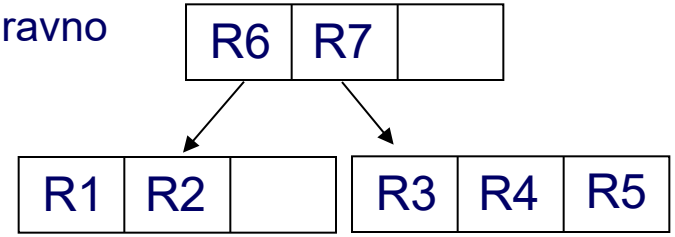
- uočiti:
  - MBR-ovi iste razine smiju se preklapati (ali čim manje - to bolje). Npr. preklapaju se R12 i R13 u prethodnom primjeru, ili R6 i R7 na slici dolje
  - ali, zapis za jedan MBR u R-stablu smije se nalaziti u samo jednom čvoru
    - ako se MBR_X i MBR_Y na *i*-toj razini preklapaju, moguće je da će neki MBR_Z na razini *i+1* biti obuhvaćen i sa MBR_X i sa MBR_Y. U takvom slučaju, MBR_Z se dodaje ili u čvor na kojeg pokazuje MBR_X ili u čvor na kojeg pokazuje MBR_Y (ne u oba)



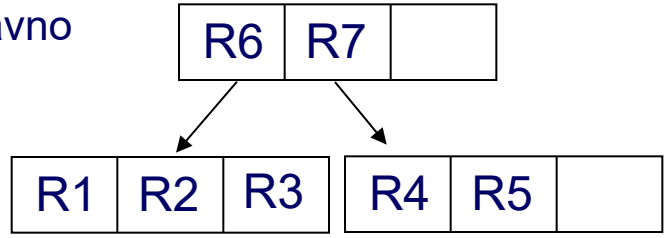
- neispravno



- ispravno

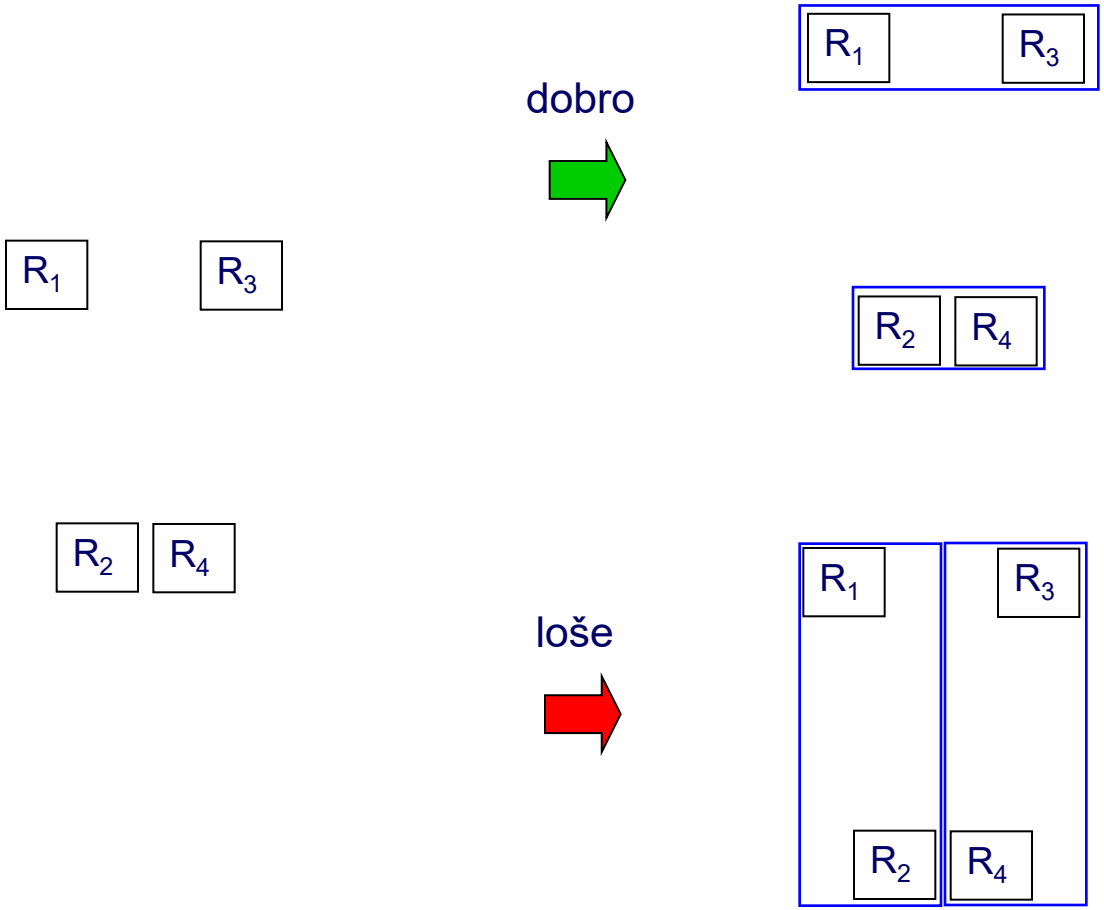


- ispravno

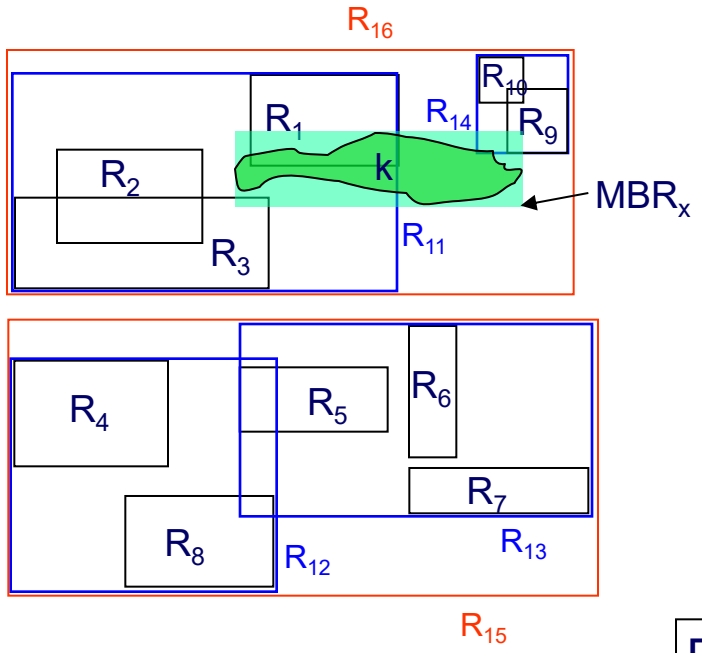


R-stablo

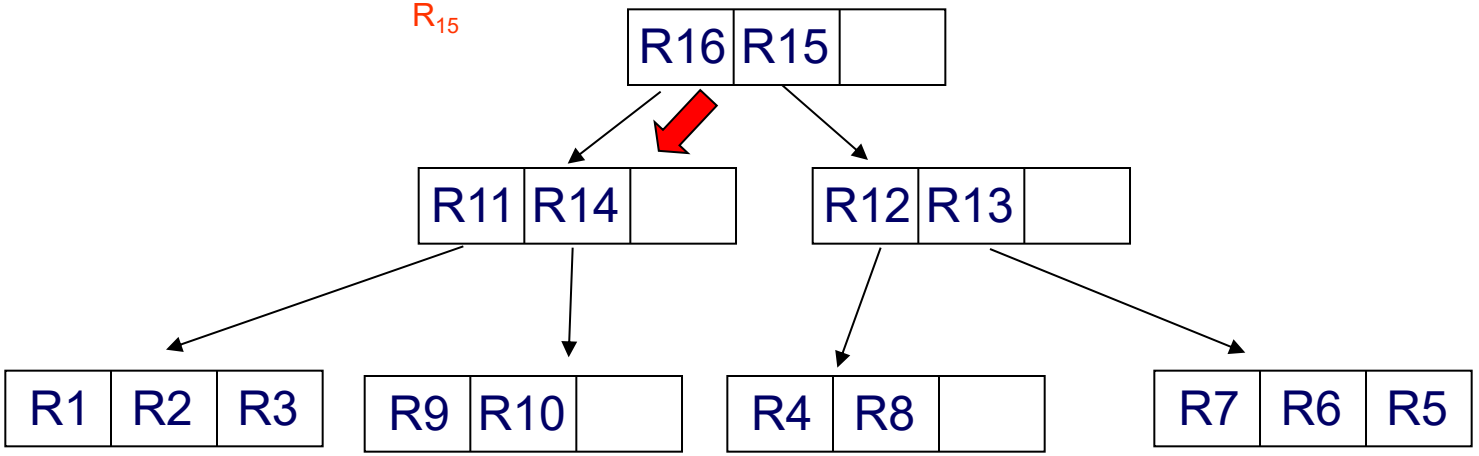
- algoritmi za grupiranje minimalnih graničnih okvira su relativno kompleksni
  - nastojati minimizirati površinu MBR-ova
- primjer dobrog i lošeg odabira



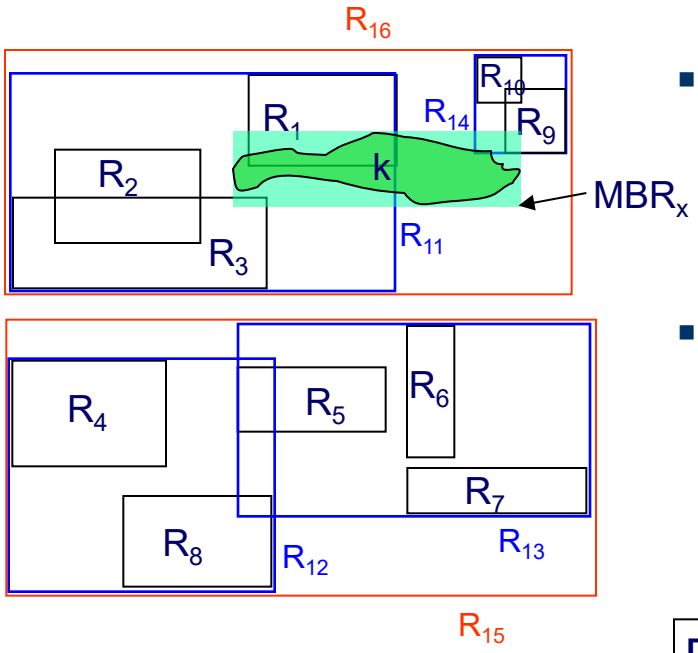
R-stablo - pretraga



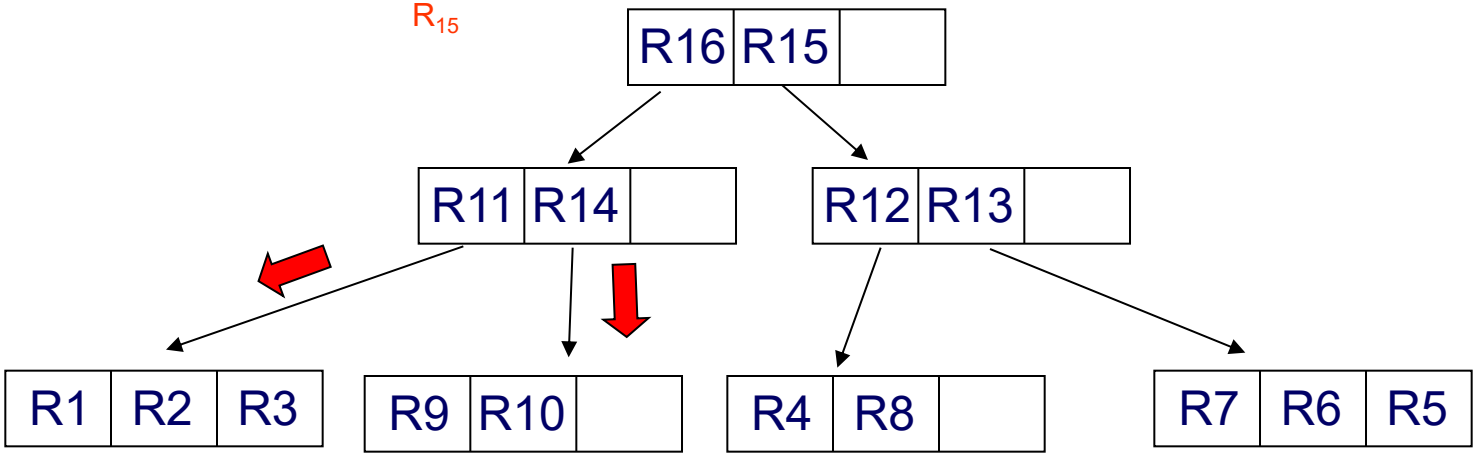
- primjer prostornog upita: dohvatiti sve objekte koji se sijeku sa zadanim objektom k (pripadni MBR objekta k je MBR_x)
- pretraga kreće od korijena
- u korijenu naći sve MBR-ove u presjeku s MBR_x i slijediti pripadne kazaljke ⇒ R16
- ponavljati na svakoj razini do lista



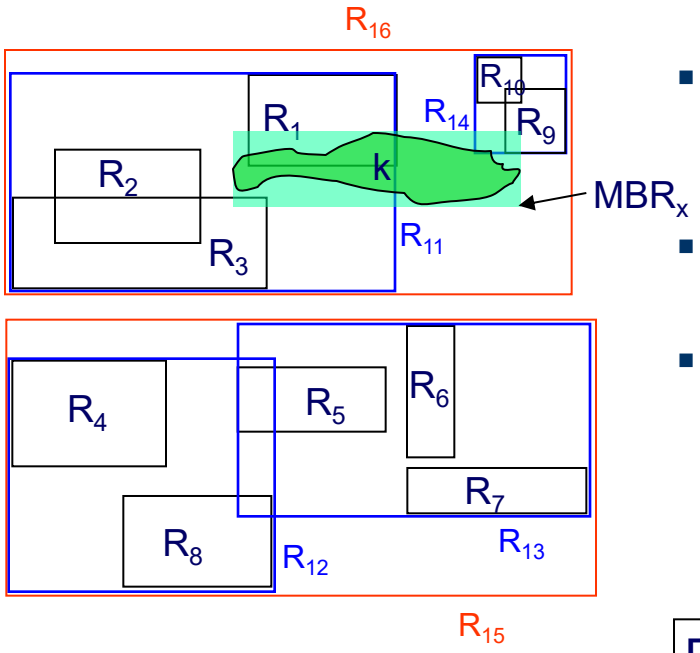
R-stablo - pretraga



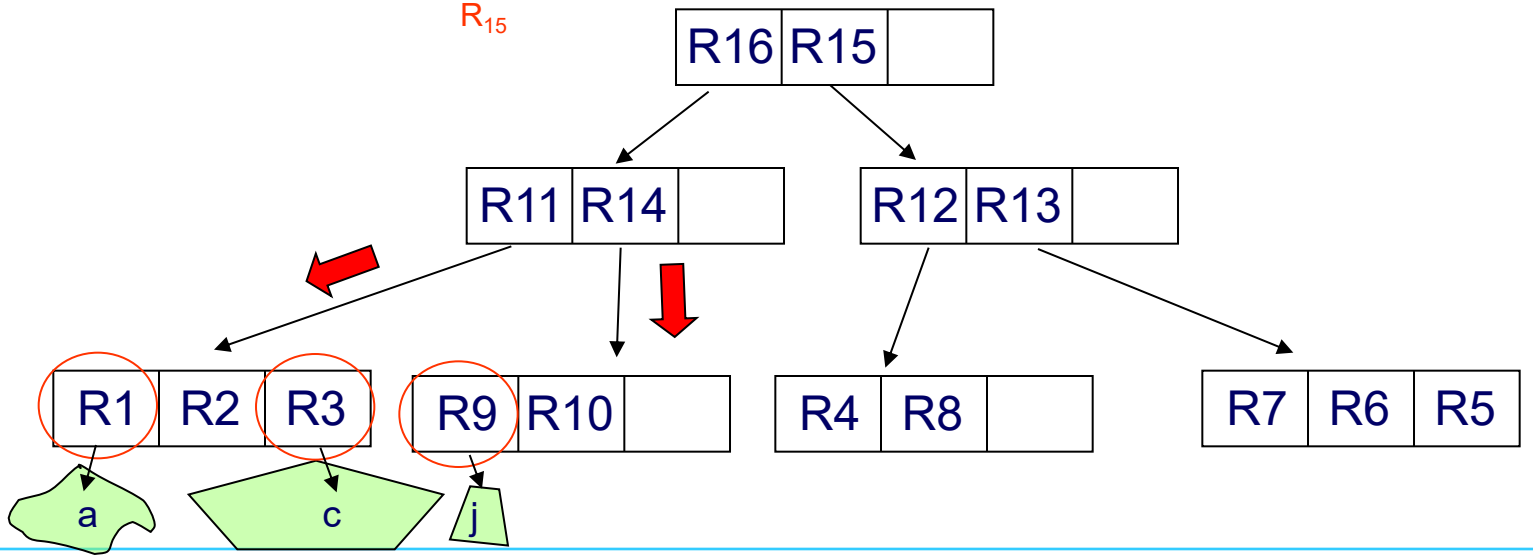
- na sljedećoj razini (razini ispod korijena) pronaći sve MBR-ove u presjeku s MBR_x i slijediti pripadne kazaljke ⇒ R11, R14
- očito, za razliku od B-stabla, moguće je da će se morati slijediti kazaljke u više podstabala



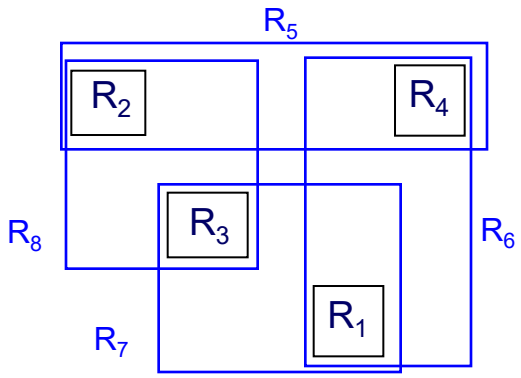
R-stablo - pretraga



- na sljedećoj razini (ovdje u **dva** lista) pronaći sve MBR-ove u presjeku s $MBR_x \Rightarrow$ R1, R3 ispod R11 i R9 ispod R14 (pripadni objekti su a, c, j)
- provjeriti koji od objekata a, c, j zaista jest u presjeku s objektom k $\Rightarrow$ a
- za razliku od B-stabla, pretragom R-stabla može se "pronaći" više MBR-ova nego treba, ali nikako ne manje (višak se eliminira naknadnom provjerom)



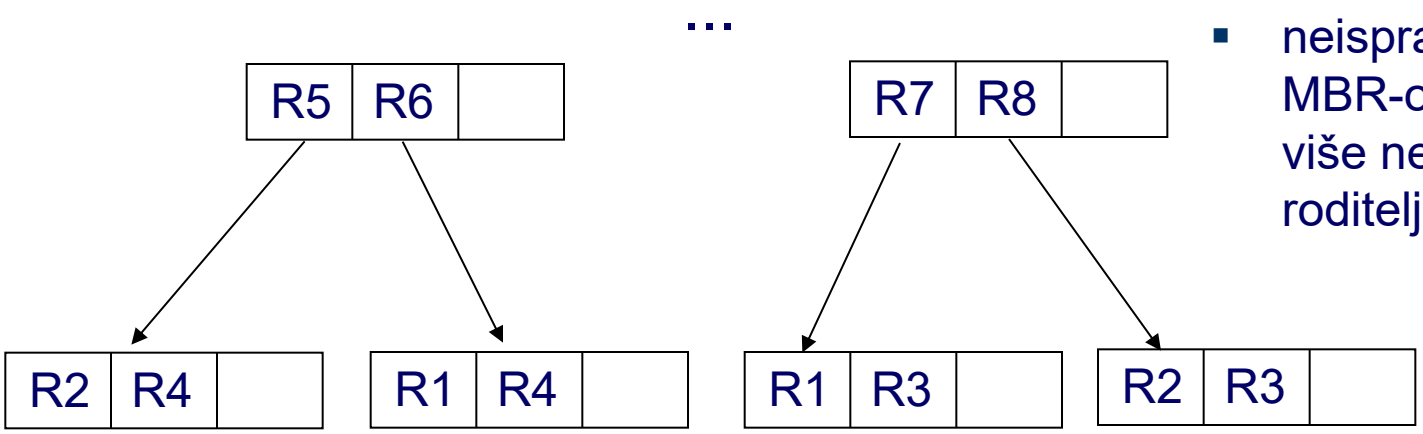
R-stablo - primjer



- svaki roditelj kompletno prekriva svoju djecu
- MBR-ovi se smiju preklapati

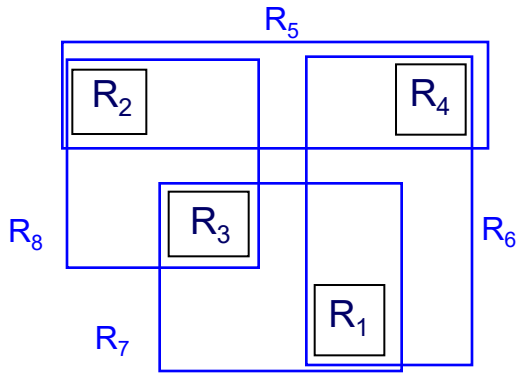
ALI

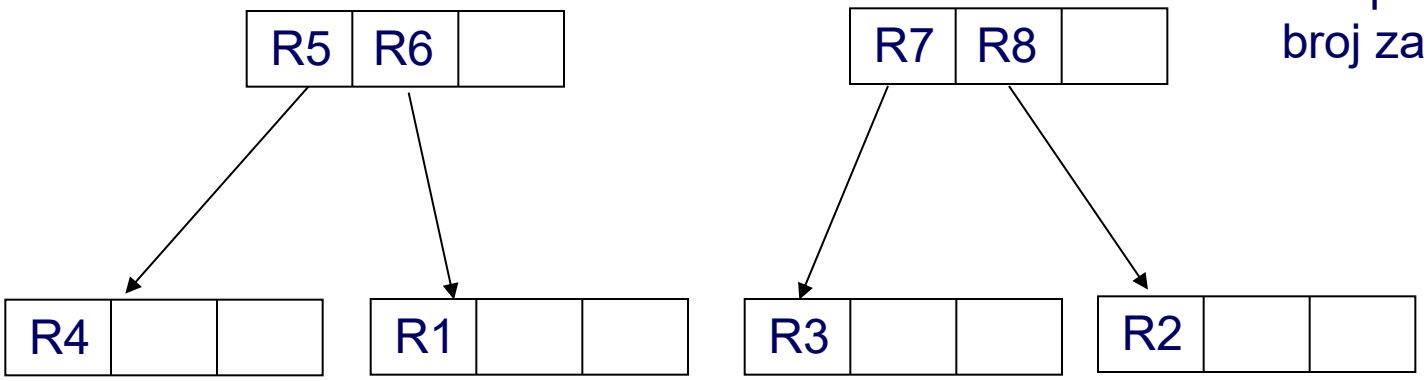
- ako se MBR-ovi preklapaju i neko dijete je prekriveno s dva (ili više MBR-ova), u stablu se dijete smije pridružiti **samo jednom** od roditelja koji ga prekrivaju



- neispravno - neki MBR-ovi su pridruženi više nego jednom roditelju

R-stablo - primjer (nastavak)

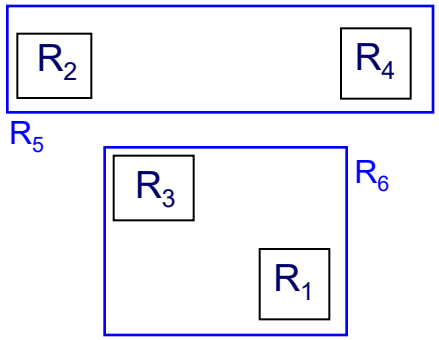




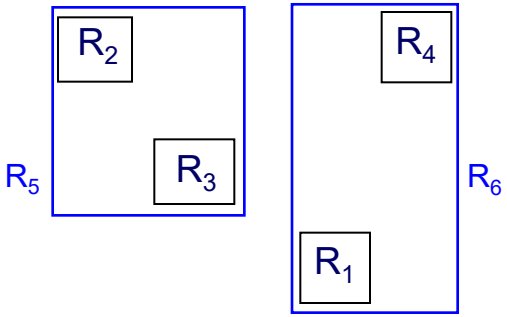
- neispravno - premalen broj zapisa u listovima

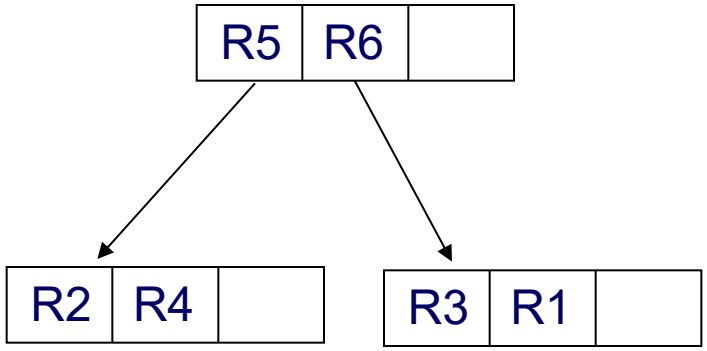
R-stablo - primjer (nastavak)

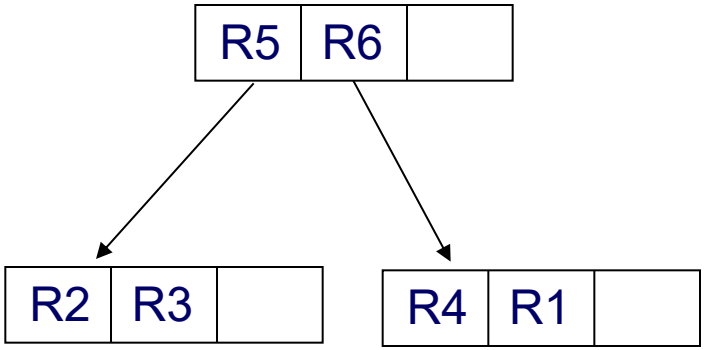
- ispravno



- ispravno







Literatura:

- H. Garcia-Molina, J. Ullman, J. Widom. Database system implementation. Prentice-Hall International, Inc. Upper Saddle River, New Jersey. 2000.
- A. Silberschatz, H. Korth, S. Sudarshan. Database system concepts, 6th ed. McGraw-Hill. 2010.
- R. Elmasri, S. Navathe. Fundamentals of database systems, 3rd ed. Addison-Wesley Publishing Company. Reading, Massachusetts. 2000.